

# Reducción de dimensionalidad



Flavio E. Spetale

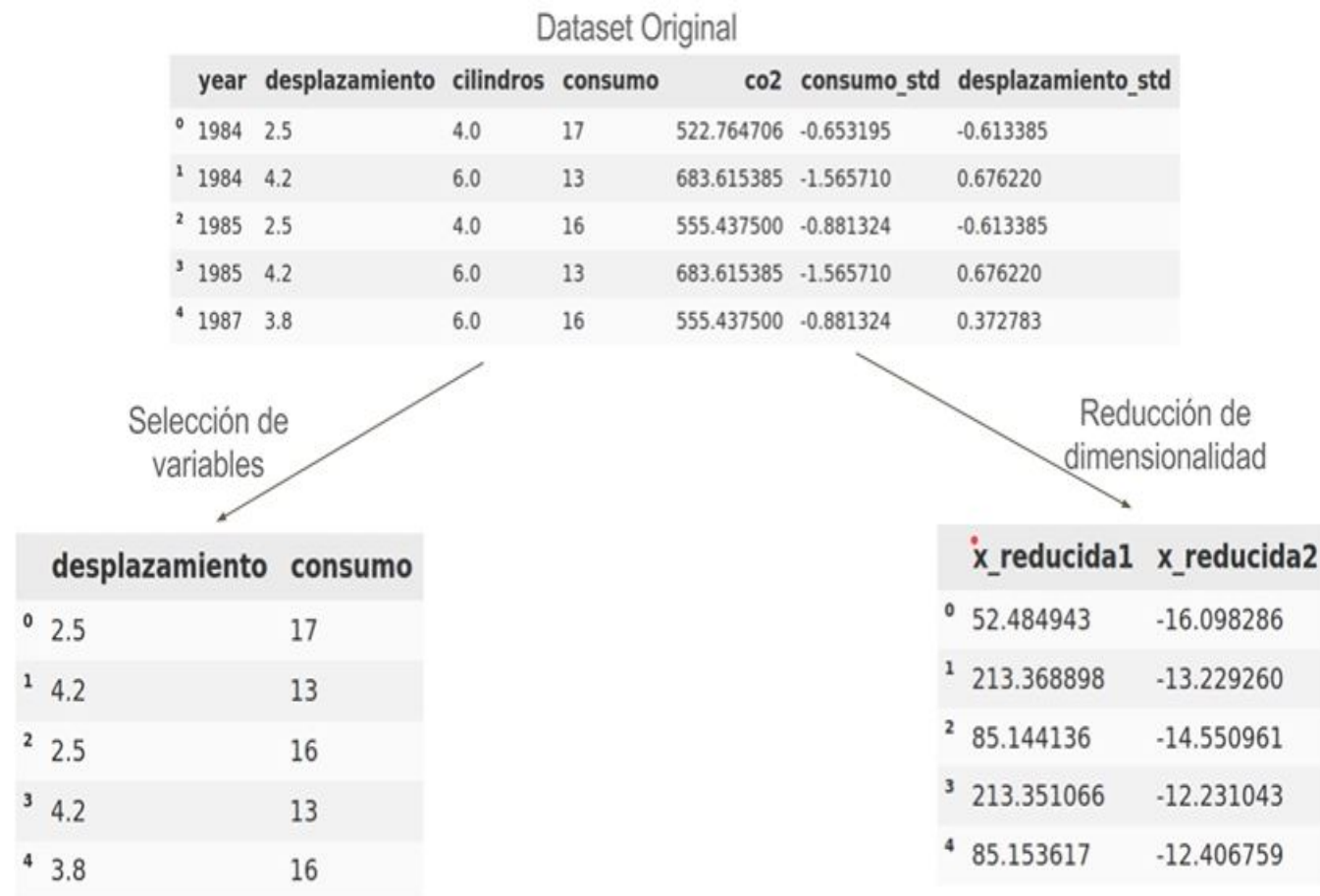


2024

# Reducción de la dimensionalidad vs Selección de Variables

Mediante la selección de variables se podría llegar a deducir que, quedándose con las columnas «desplazamiento» y «consumo», se pueden obtener modelos predictores de alto rendimiento. De esta manera se obvia el resto de variables. Por tanto, no hay pérdida de la significancia de las variables.

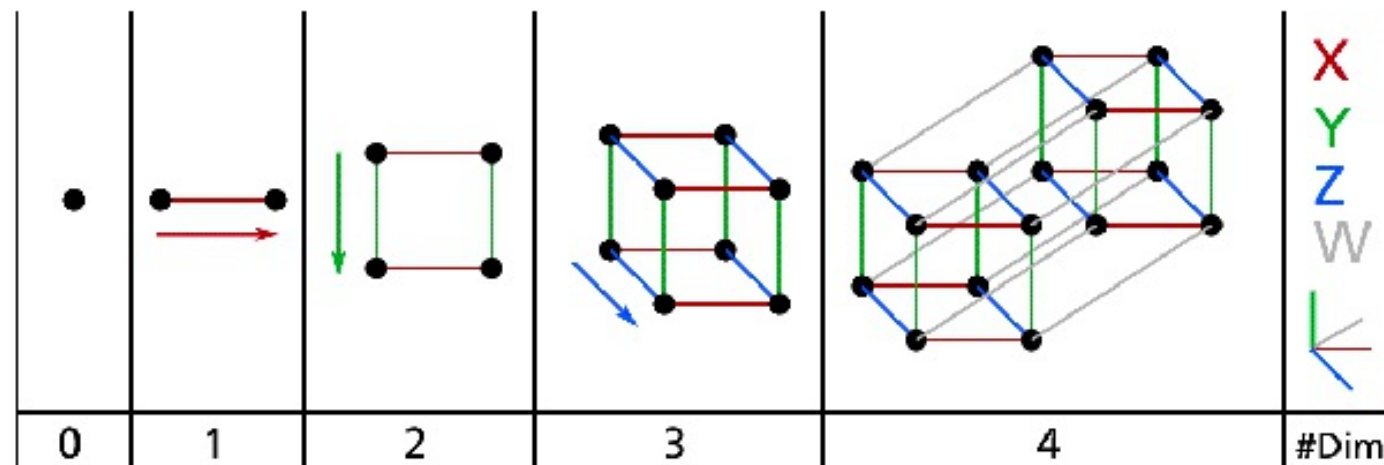
Las técnicas de reducción de dimensionalidad aplicadas a este conjunto de datos, en cambio, producen algo como lo mostrado en la flecha de la derecha: dos columnas (variables) a las que se ha dado el nombre de «x\_reducida1» y «x\_reducida2». En este ejemplo se empleó PCA para reducir el conjunto de datos a esas 2 «componentes» llamadas «x\_reducida1» y «x\_reducida2».



# Reducción de la dimensionalidad

La mayoría de los profesionales en el campo del Data Science coinciden en que, para la construcción de un modelo de data mining bueno, se debe invertir alrededor de un 75% del tiempo y esfuerzo en la fase de preprocesado de los datos.

Muchos problemas de aprendizaje automático involucran miles o incluso millones de características para cada instancia de entrenamiento. No sólo todas estas características hacen el entrenamiento es extremadamente lento, pero también pueden hacer que sea mucho más difícil encontrar una buena solución, como verá. Este problema a menudo se conoce como *la maldición de la dimensionalidad*.



# La maldición de la dimensionalidad

En aprendizaje computacional, el número de dimensiones se puede equiparar al número de variables o características que estemos utilizando.

La maldición de la dimensión se *manifiesta* de dos maneras:

- La distancia media entre los datos aumenta con el número de dimensiones.
- La variabilidad de la distancia disminuye exponencialmente con el número de dimensiones (éste es el verdadero problema).

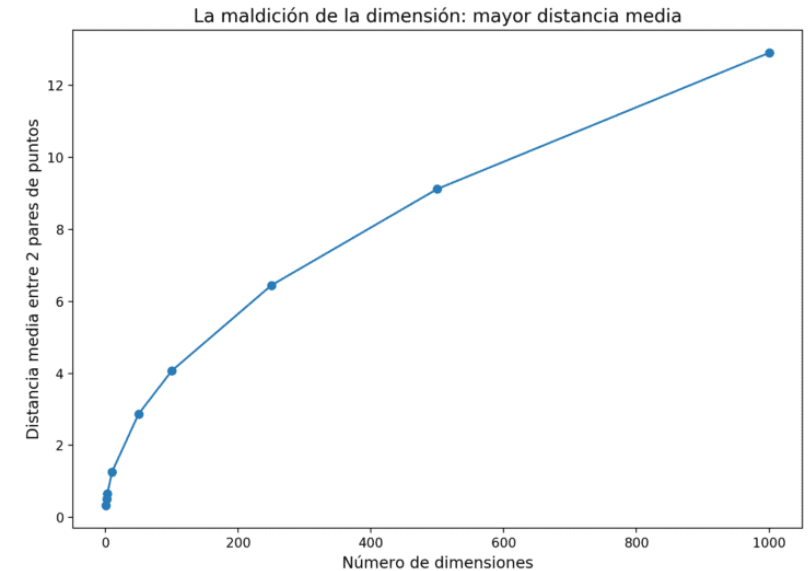
Estamos tan acostumbrados a vivir en tres dimensiones que nuestra intuición nos falla cuando tratamos de imaginar un espacio de altas dimensiones. Incluso un hipercubo 4D básico es increíblemente difícil de imaginar en nuestra mente, ni hablar de una elipsoide de 200 dimensiones llevada a un espacio de 1.000 dimensiones.

***Se comportan de manera muy diferente en el espacio de alta dimensión.***

# La maldición de la dimensionalidad

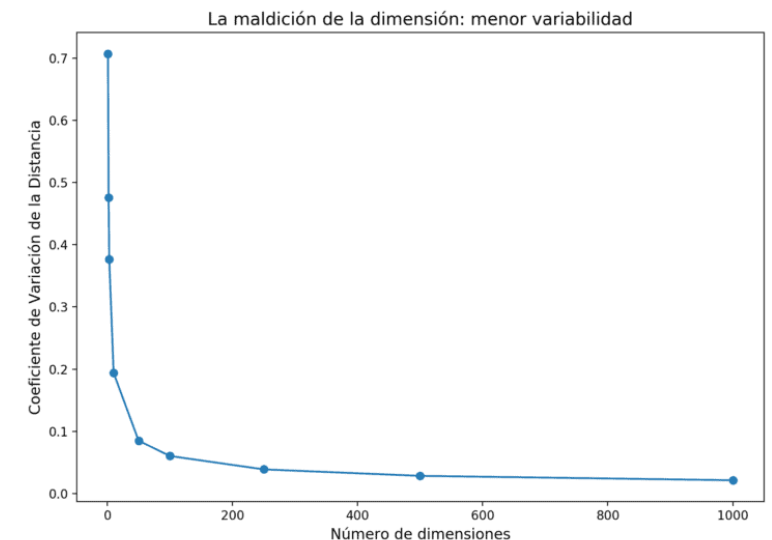
## *La distancia media aumenta con el número de dimensiones*

A medida que el número de dimensiones aumenta, la distancia media entre ellos también aumenta. Esto en sí mismo no es muy problemático para el aprendizaje automático.



## *La variabilidad disminuye exponencialmente*

Lo verdaderamente importante es que a medida que el número de dimensiones aumenta ... ¡la variabilidad de las distancias entre los datos disminuye exponencialmente!



# La maldición de la dimensionalidad

Como resultado, los conjuntos de datos de alta dimensión corren el riesgo de ser muy escasos: es probable que la mayoría de las instancias de entrenamiento estén lejos unas de otras. *Esto significa que una nueva instancia probablemente estará lejos de cualquier instancia de entrenamiento, lo que hace que las predicciones sean mucho menos confiables que en dimensiones más bajas, ya que se basarán en extrapolaciones mucho mayores.*

Cuanto más dimensiones tenga el conjunto de entrenamiento, mayor será el riesgo de sobreajuste.

En teoría, una solución a la maldición de la dimensionalidad podría ser aumentar el tamaño del conjunto de entrenamiento para alcanzar una densidad suficiente de instancias de entrenamiento. **Desafortunadamente, en la práctica, la cantidad de instancias de entrenamiento requeridas para alcanzar una determinada densidad crece exponencialmente con la cantidad de dimensiones.**

**Ej:** Con solo 100 características [0-1], en el problema MNIST (dígitos), necesitaría más instancias de entrenamiento que átomos en el universo observable para que las instancias de entrenamiento estén dentro de 0.1 entre sí en promedio, suponiendo que pudieran distribuirse uniformemente en todas las dimensiones.

# La maldición de la dimensionalidad

La maldición de la dimensión sólo afecta a las técnicas basadas en distancias.

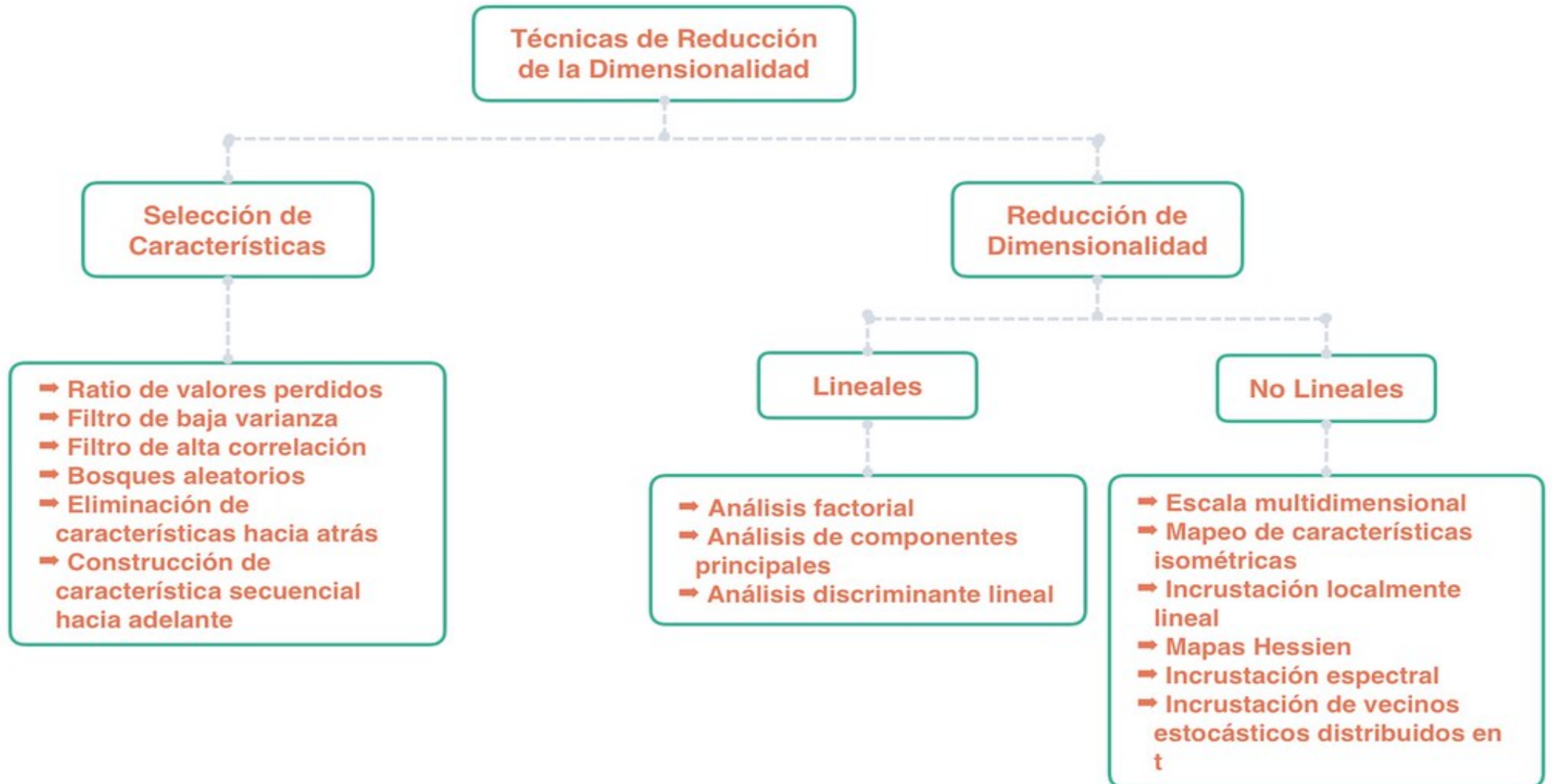
Estas son algunas de las técnicas de aprendizaje automático afectadas por la maldición de la dimensión:

- **Agrupamiento (clustering)**: para encontrar automáticamente grupos en los datos
- **K vecinos más cercanos (KNN)**: ofrece una predicción combinando los resultados de instancias similares.
- Técnicas de **detección de anomalías** basadas en distancias tales como Local Outlier Factor.

Para mitigar el efecto de la maldición de la dimensión tenemos dos opciones:

- Reducir el número de dimensiones
- Aumentar la cantidad de datos

# Reducción de la dimensionalidad



# Proyección

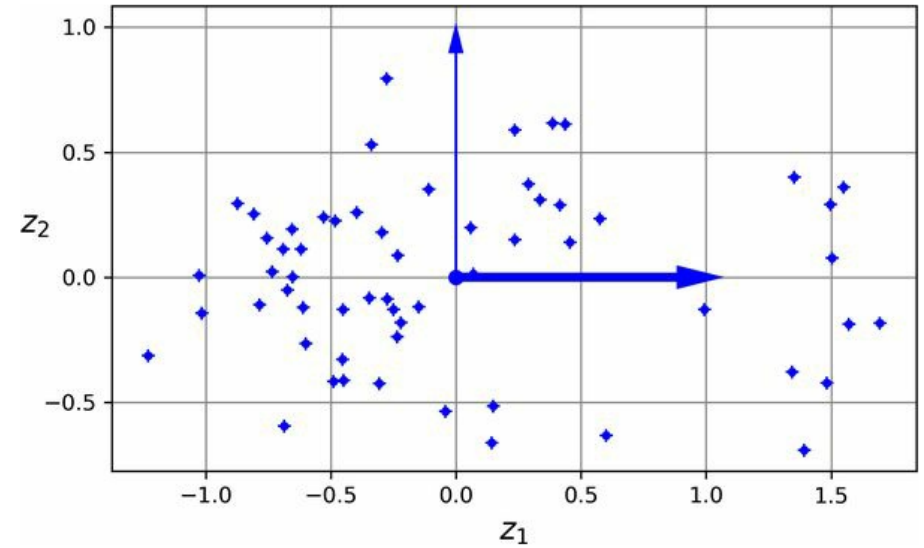
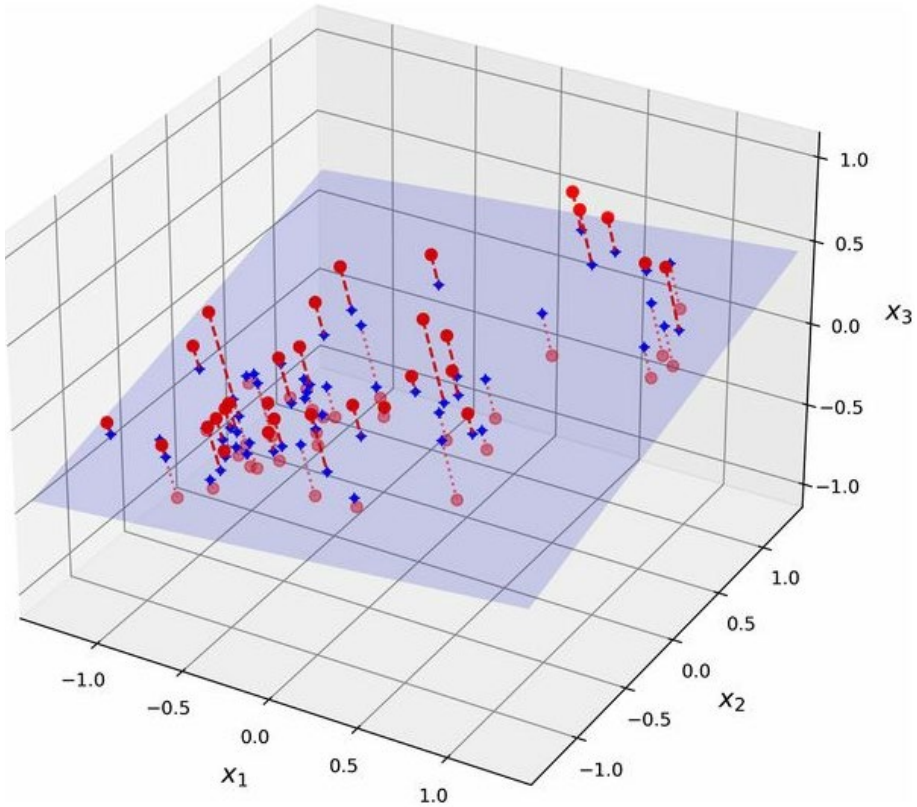
En la mayoría de los problemas del mundo real, las instancias de capacitación no se distribuyen uniformemente en todas las dimensiones.

Muchas características son casi constantes, mientras que otras están altamente correlacionadas.

Como resultado, todas las instancias de entrenamiento se encuentran dentro (o cerca de) un subespacio de dimensiones mucho más bajas del espacio de alta dimensión.

**Proyección:** Es una técnica que consiste en proyectar cada punto de datos que se encuentra en una dimensión alta, en un subespacio adecuado de menor dimensión de manera que se preserven aproximadamente las distancias entre los puntos.

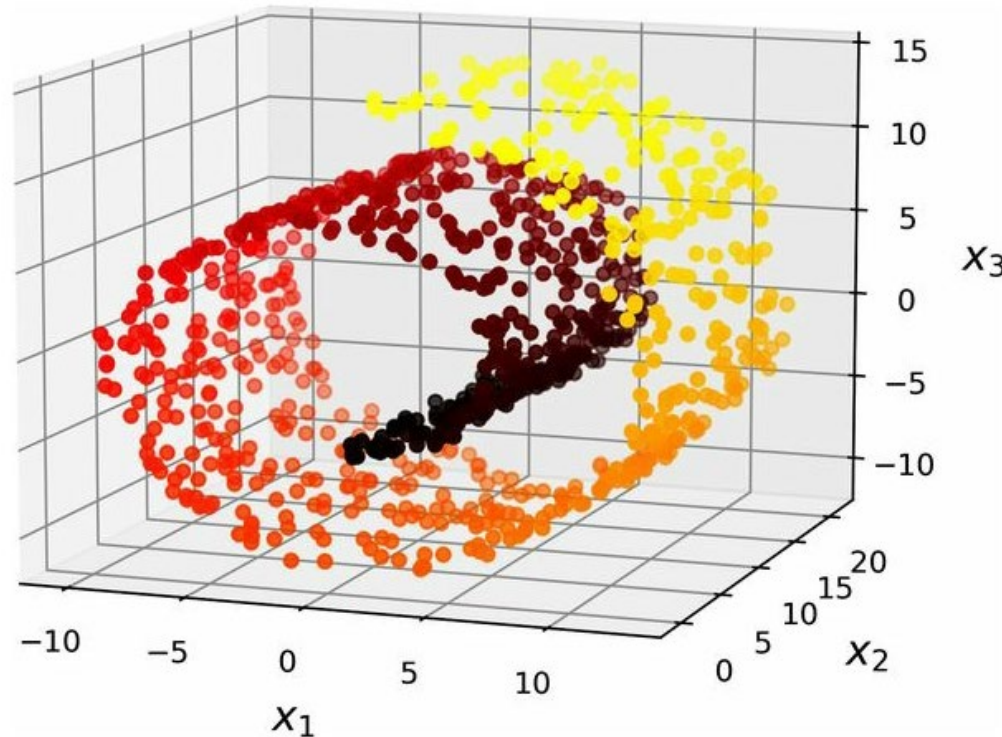
# Proyección



Acabamos de reducir la dimensionalidad del conjunto de datos de 3D a 2D.  
Los ejes  $z_1$  y  $z_2$  corresponden a las nuevas características y son las coordenadas de las proyecciones en el plano.

# Manifolding Learning

Sin embargo, la proyección no siempre es el mejor enfoque para la reducción de dimensionalidad. En muchos casos, el subespacio puede girar y girar, como en el famoso conjunto de datos sintético del rollo suizo.

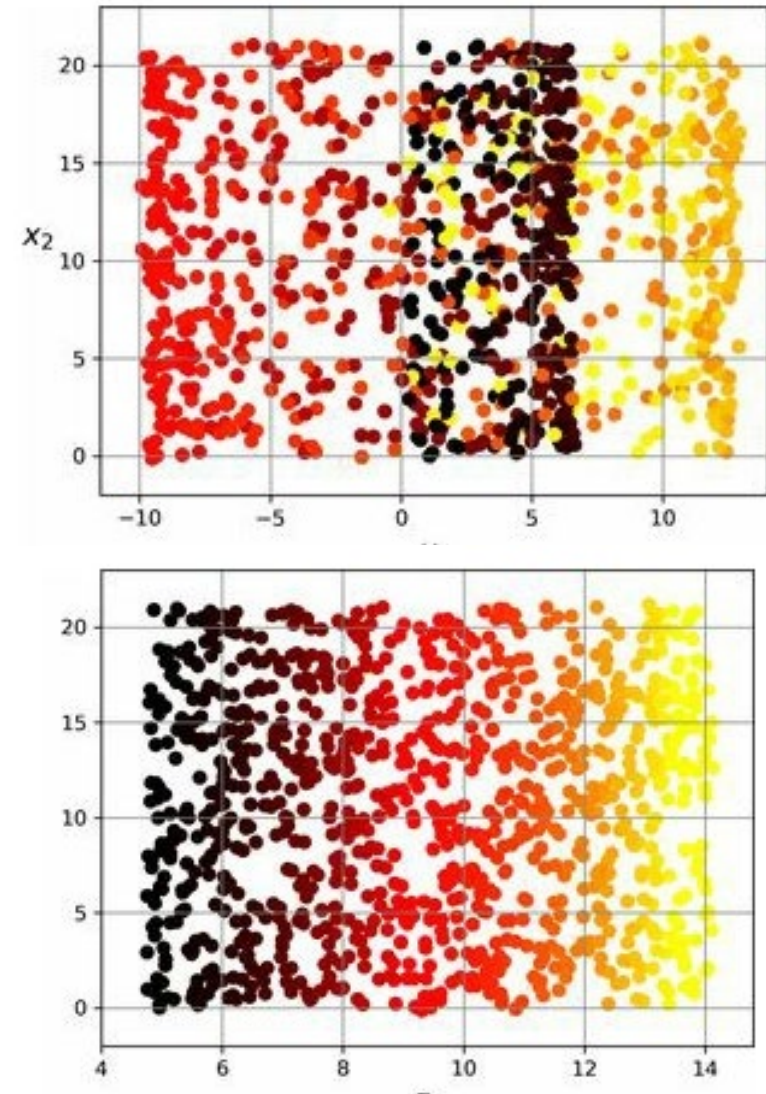


# Manifolding Learning

Simplemente proyectar sobre un plano (por ejemplo, dejando caer  $x_3$ ) aplastaría diferentes capas del rollo suizo

Lo que probablemente desee es desenrollar el rollo suizo para obtener el conjunto de datos 2D.

El rollo suizo es un ejemplo de *manifolding 2D*



# Manifolding Learning

Un manifolding 2D es una forma 2D de doblar y torcer un espacio de dimensiones superiores.

## **Definición**

*Un manifolding  $d$ -dimensional es parte de un espacio  $n$ -dimensional (con  $d < n$ ) que localmente se asemeja a un hiperplano  $d$ -dimensional.*

En el caso del rollo suizo,  $d = 2$  y  $n = 3$ . Localmente se parece a un plano 2D, pero está enrollado en la tercera dimensión.

Los algoritmos de reducción de dimensionalidad que funcionan modelando la variedad en la que se encuentran las instancias de entrenamiento se los conoce como aprendizaje múltiple (manifolding learning).

Se basan en la suposición múltiple, también llamada *hipótesis de la variedad*, que sostiene que la mayoría de los conjuntos de datos de alta dimensión del mundo real se encuentran cerca de una variedad de dimensiones mucho más bajas.

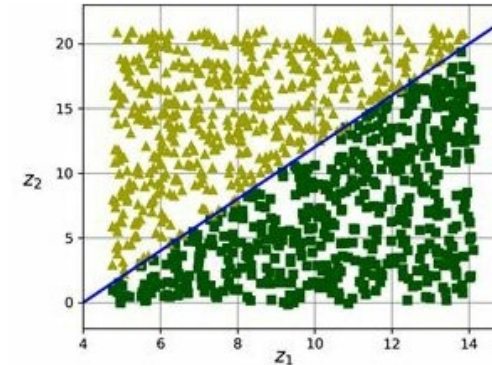
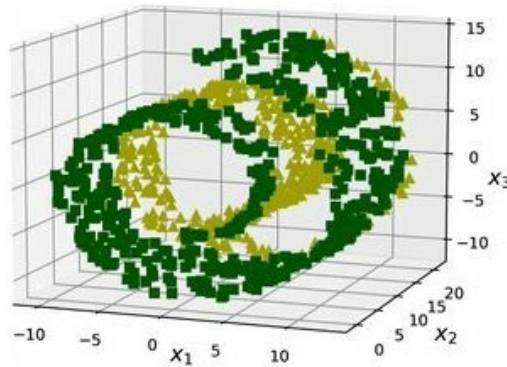
Esta suposición se observa muy a menudo empíricamente.

# Manifolding Learning

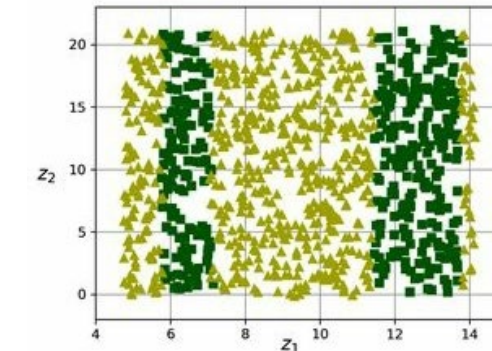
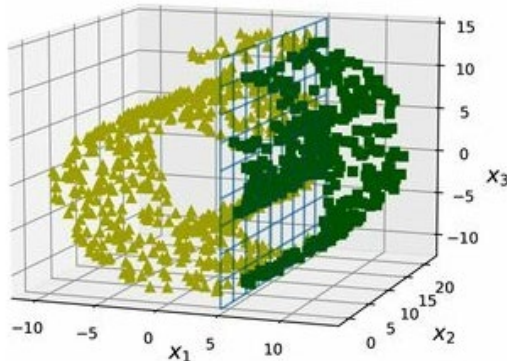
El supuesto múltiple suele ir acompañado de otro supuesto implícito: que la tarea en cuestión (por ejemplo, clasificación o regresión) será más simple si se expresa en el espacio de dimensiones inferiores de la variedad.

Ejemplo: Rollo suizo con dos clases

**Fácil**



**Difícil**

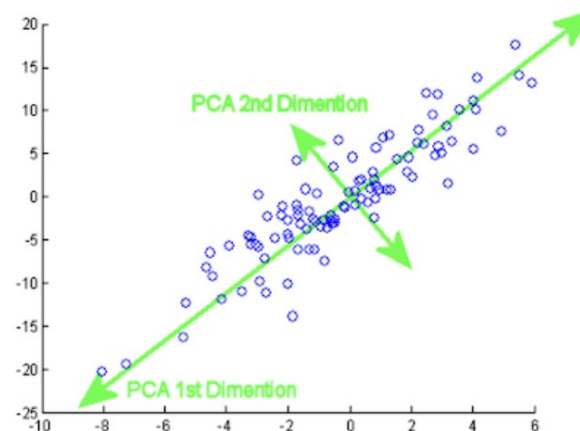


# Análisis de componentes principales (PCA)

PCA es un método que se encarga de encontrar los componentes principales de un conjunto de datos mediante técnicas de **factorización matricial**. Es decir, es un método estadístico que permite simplificar la complejidad de espacios muestrales con muchas dimensiones a la vez que conserva su información.

PCA pertenece a la familia de técnicas conocida como aprendizaje no supervisado.

El método de PCA permite por lo tanto “condensar” la información aportada por múltiples variables en solo unas pocas componentes. Esto lo convierte en un método muy útil de aplicar previa utilización de otras técnicas estadísticas tales como regresión, clustering, etc. Aun así no hay que olvidar que sigue siendo necesario disponer del valor de las variables originales para calcular las componentes.



# Análisis de componentes principales (PCA)

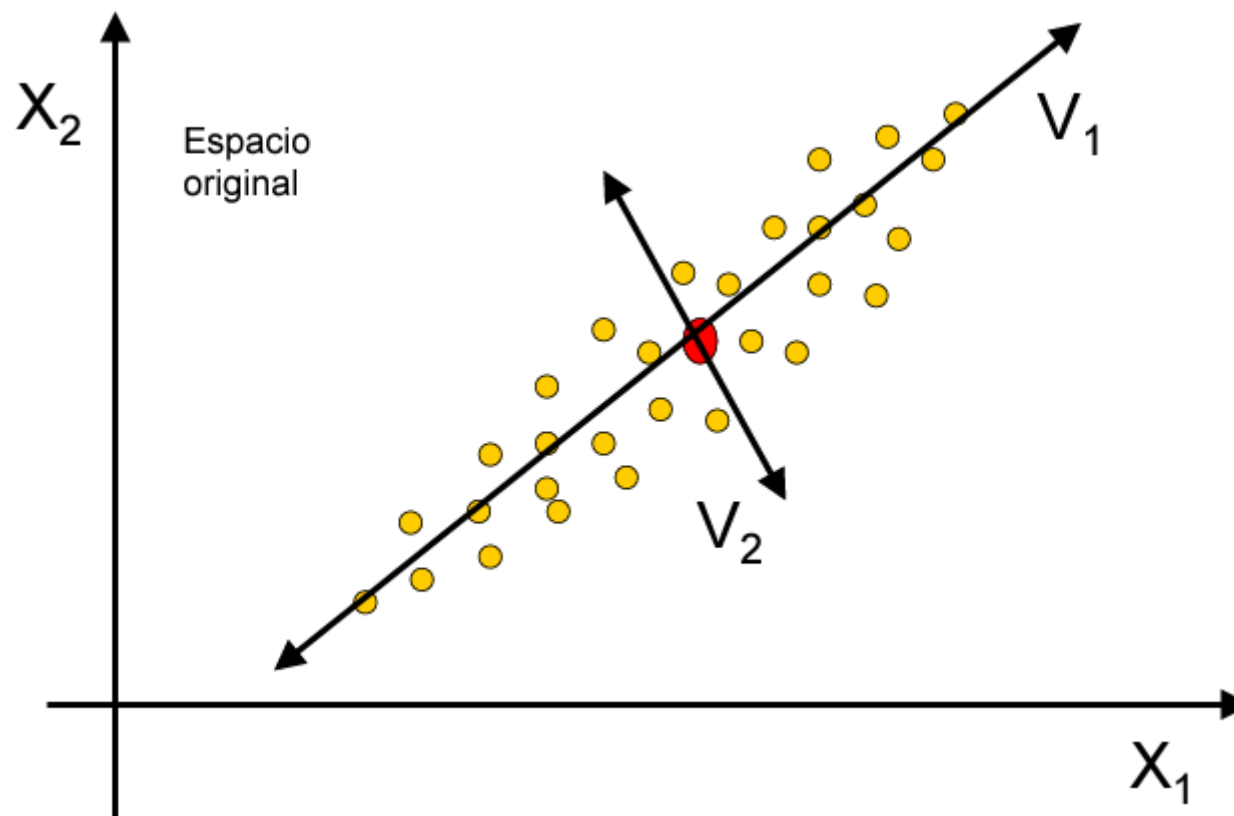
## ***¿Qué son los componentes principales?***

Los componentes principales de un conjunto de datos (o de una matriz, que viene a ser lo mismo) son aquellas direcciones en las cuáles la varianza de los elementos del conjunto de datos es máxima. Es decir, son aquellas direcciones en las cuales los elementos están más separados los unos de los otros.

Al descomponer el conjunto de datos en N componentes principales (donde N es el número de dimensiones que queremos) reducimos la dimensionalidad del conjunto de datos pero conservando la mayor varianza posible y por lo tanto conservando la mayor parte de la información en dicho conjunto de datos.

***Técnica exploratoria que procura hallar aquellas combinaciones (lineales) de las variables originales que maximizan la varianza (información).***

# PCA: Intuición



# Análisis de componentes principales (PCA)

## Finalidad de las Componentes Principales

- Hallar variables latentes (componentes o factores).
- Reducir la dimensión del problema.
- Eliminar redundancias de la información.
- Obtener una representación gráfica de información multidimensional.

# PCA: Matriz de varianzas-covarianzas empírica

Matriz de datos centrada

$$X =$$

|          | 1         | ... | j         | ... | p         |
|----------|-----------|-----|-----------|-----|-----------|
| 1        | $X_{1,1}$ |     | $X_{1,j}$ |     | $X_{1,p}$ |
| $\vdots$ |           |     |           |     |           |
| i        | $X_{i,1}$ |     | $X_{i,j}$ |     | $X_{i,p}$ |
| $\vdots$ |           |     |           |     |           |
| n        | $X_{n,1}$ |     | $X_{n,j}$ |     | $X_{n,p}$ |



Matriz de varianzas-covarianzas

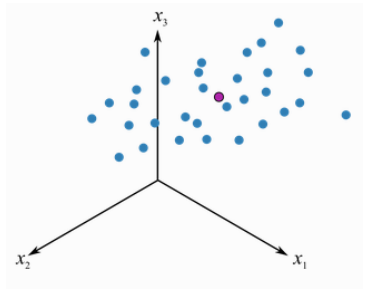
|          | $X_1$                  | ...      | $X_j$                  | ...      | $X_p$                  |
|----------|------------------------|----------|------------------------|----------|------------------------|
| $X_1$    | $\text{Var}(X_1)$      | ...      | $\text{Cov}(X_1, X_j)$ | ...      | $\text{Cov}(X_1, X_p)$ |
| $\vdots$ | $\vdots$               | $\vdots$ | $\vdots$               | $\vdots$ | $\vdots$               |
| $X_j$    | $\text{Cov}(X_j, X_1)$ | ...      | $\text{Var}(X_j)$      | ...      | ...                    |
| $\vdots$ | $\vdots$               | $\vdots$ | $\vdots$               | $\vdots$ | $\vdots$               |
| $X_p$    | $\text{Cov}(X_p, X_1)$ | ...      | ...                    | ...      | $\text{Var}(X_p)$      |

$S = \{ s_{i,j} \}$

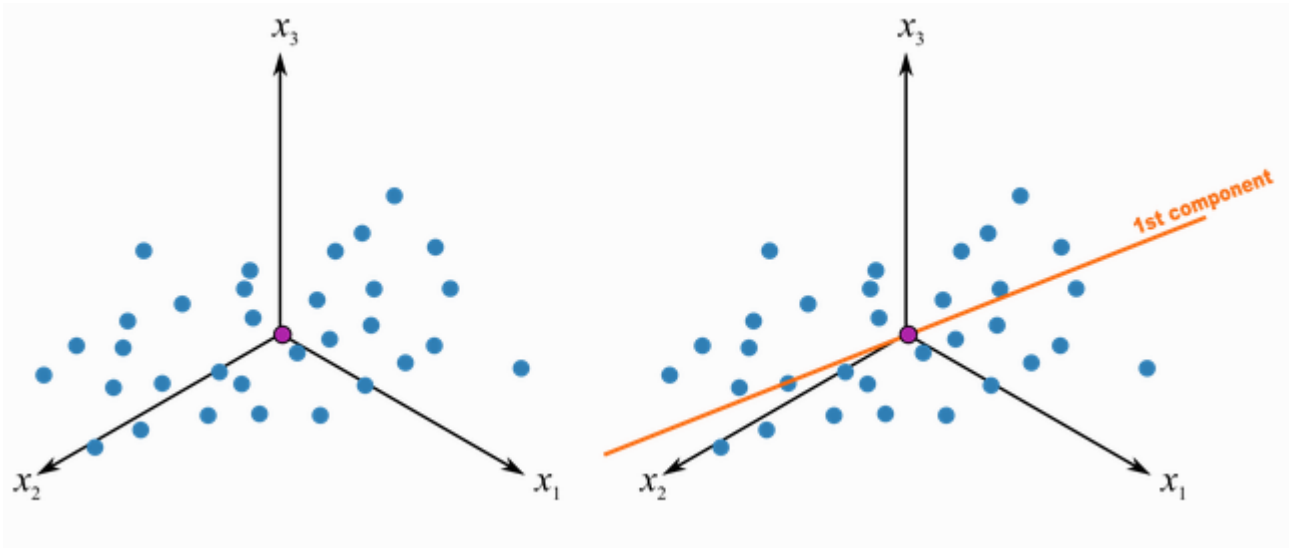
$$s_{i,j} = \frac{1}{n} \sum_{\ell=1}^n (x_{\ell i} - \bar{x}_i)(x_{\ell j} - \bar{x}_j)$$

# Análisis de componentes principales (PCA)

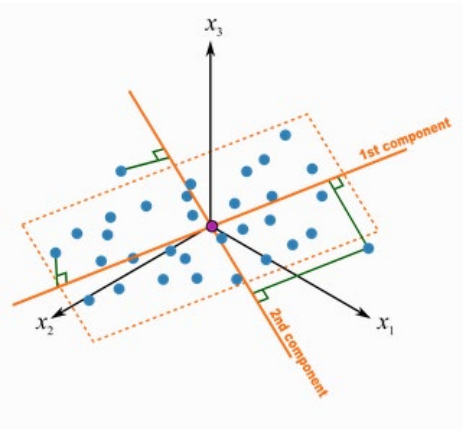
Explicación geométrica para datos en 3 dimensiones  $x_1$ ,  $x_2$ ,  $x_3$



el punto violeta representa la media de las  $N$  observaciones en azul



Los ejes se centran en la media. La primera componente principal ( $z_1$ ), pasará en la dirección de máxima varianza de las proyecciones de las observaciones



Luego, se calcula la segunda ( $z_2$ ), de forma que también pase por el origen y sea perpendicular a  $z_1$ , maximizando la varianza de proyección sobre esta nueva dirección.  $z_1$  y  $z_2$  definirán un plano, que constituye la mejor representación de los datos en una dimensión menor.

# PCA: Primera componente

|     | 1         | ... | j         | ... | p         |
|-----|-----------|-----|-----------|-----|-----------|
| 1   | $X_{1,1}$ |     | $X_{1,j}$ |     | $X_{1,p}$ |
| ... |           |     |           |     |           |
| i   | $X_{i,1}$ |     | $X_{i,j}$ |     | $X_{i,p}$ |
| ... |           |     |           |     |           |
| n   | $X_{n,1}$ |     | $X_{n,j}$ |     | $X_{n,p}$ |

$$\begin{pmatrix} Y_{11} \\ \vdots \\ Y_{n1} \end{pmatrix} = a_{11} \begin{pmatrix} x_{11} \\ \vdots \\ x_{n1} \end{pmatrix} + a_{21} \begin{pmatrix} x_{12} \\ \vdots \\ x_{n2} \end{pmatrix} + \cdots + a_{p1} \begin{pmatrix} x_{1p} \\ \vdots \\ x_{np} \end{pmatrix}$$

Componente

$$Y_1 = \sum_{j=1}^p a_{j1} X_j = \mathbf{X} \mathbf{a}_1$$

Autovector 1

Tal que

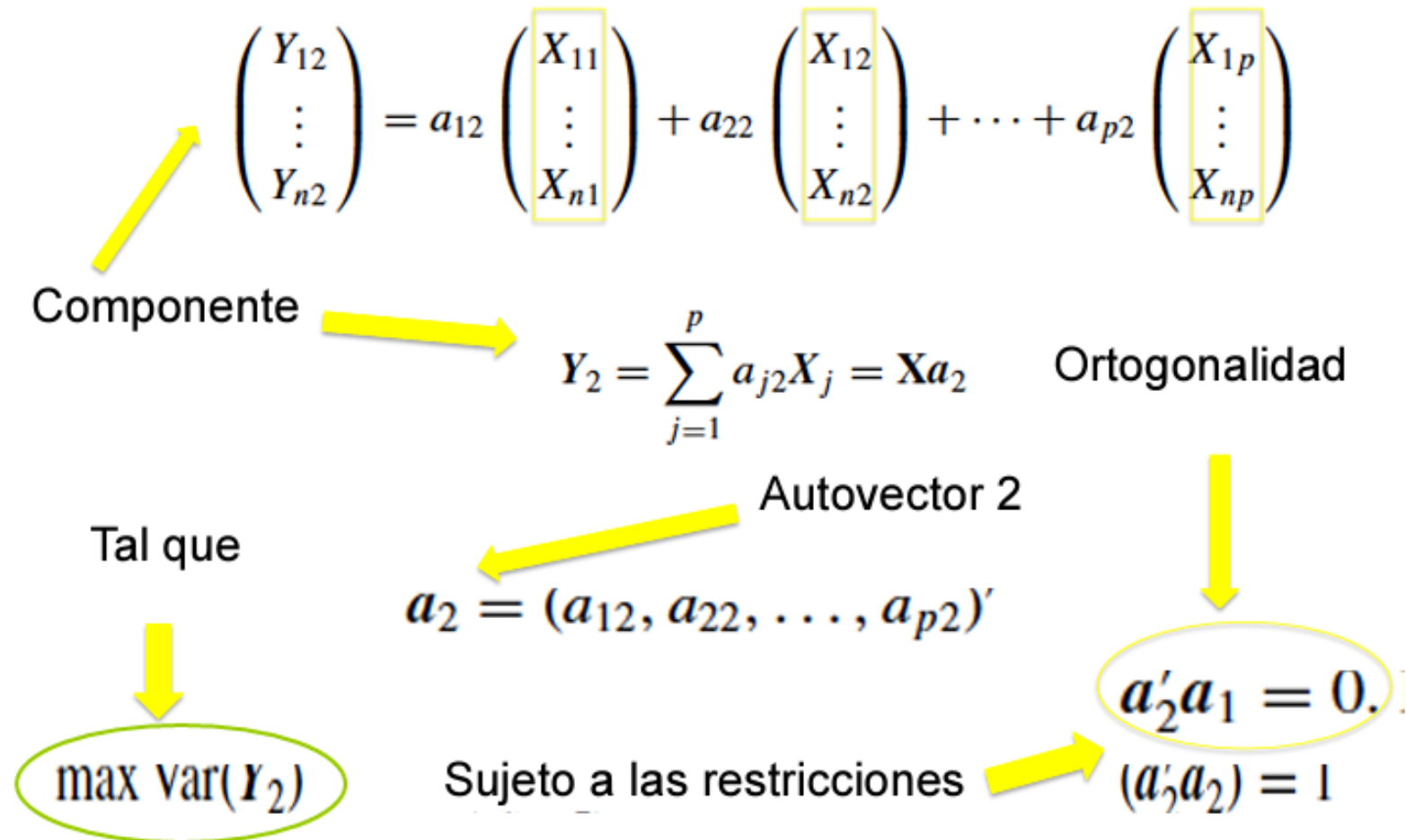
$$\mathbf{a}_1 = (a_{11}, a_{21}, \dots, a_{p1})'$$

$$\max \text{Var}(Y_1)$$

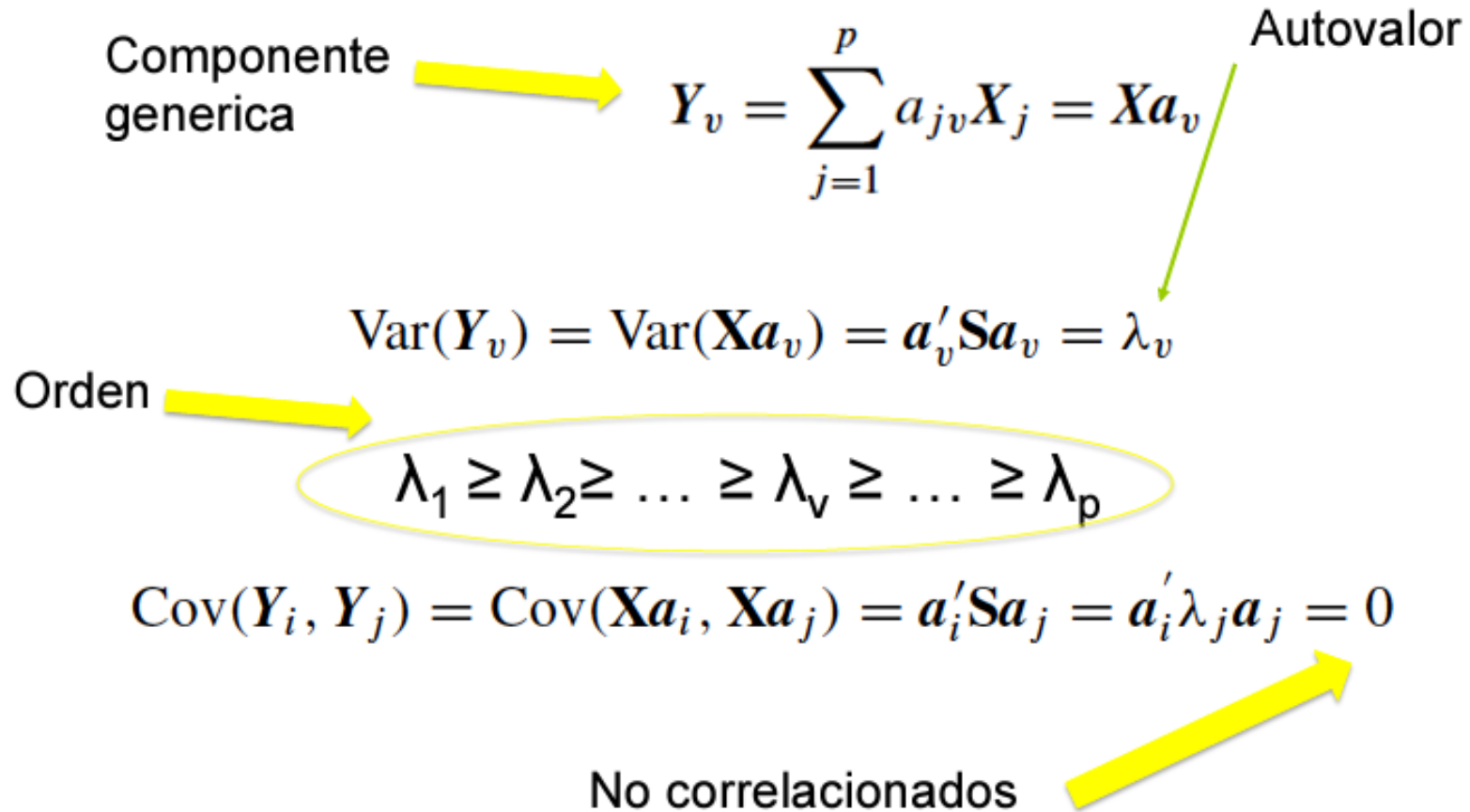
Sujeto a la restricción

$$\mathbf{a}_1' \mathbf{a}_1 = 1$$

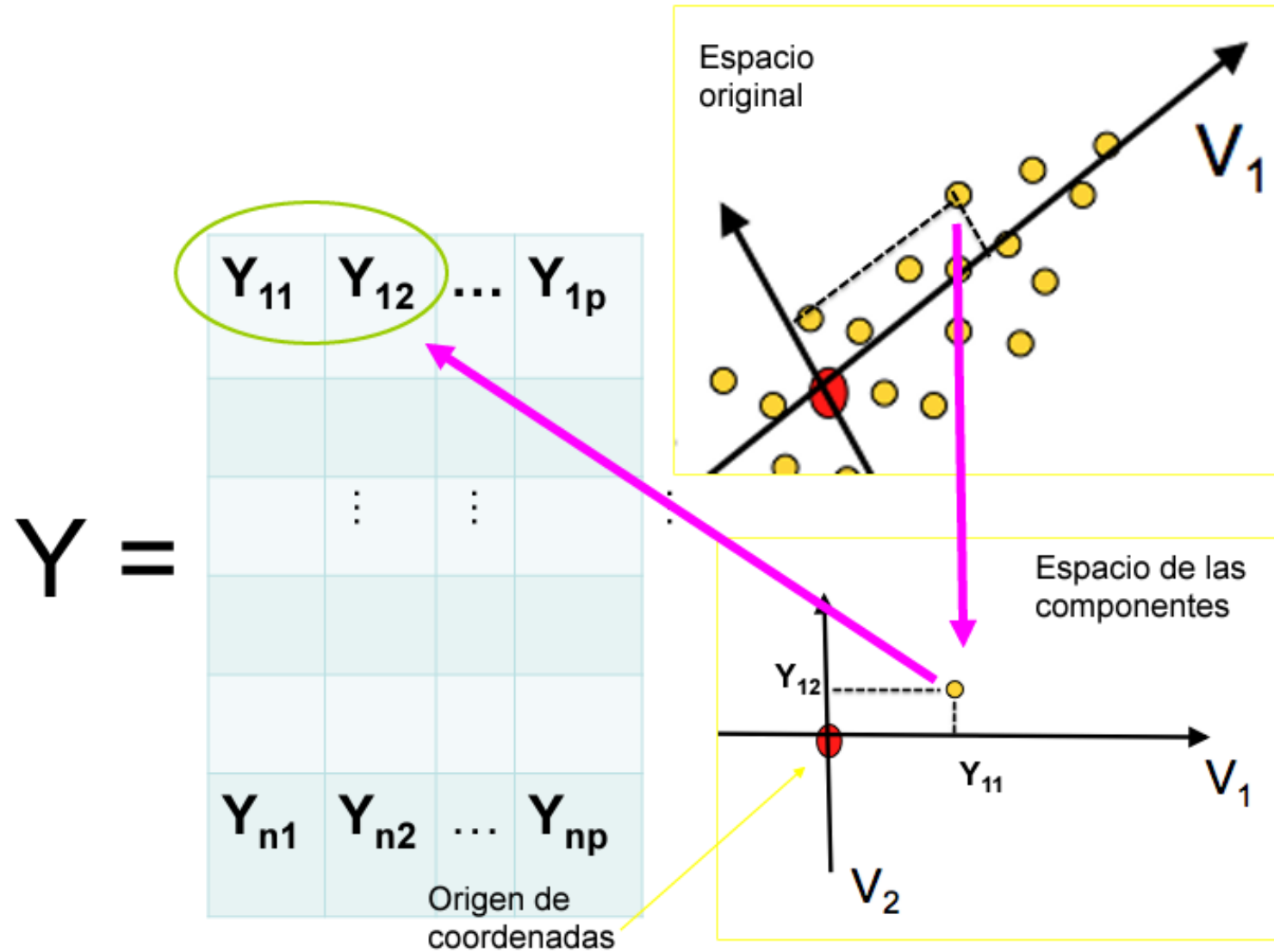
# PCA: Segunda componente



# PCA: Propiedades de las componentes



# PCA: Scores



# PCA: Consideraciones importantes

## *Escalado de las variables*

El proceso de PCA identifica aquellas direcciones en las que la varianza es mayor. Como la varianza de una variable se mide en su misma escala elevada al cuadrado, si antes de calcular las componentes no se estandarizan todas las variables para que tengan media 0 y desviación estándar 1, aquellas variables cuya escala sea mayor dominarán al resto.

## *Reproducibilidad de las componentes*

El proceso de PCA genera siempre las mismas componentes principales independientemente del software utilizado, i.e., el valor de los autovalores resultantes es el mismo. La única diferencia que puede darse es que el signo de todos los autovalores esté invertido.

## *Influencia de outliers*

Al trabajar con varianzas, PCA es altamente sensible a outliers, por lo que es altamente recomendable estudiar si los hay. La detección de valores atípicos con respecto a una determinada dimensión es algo relativamente sencillo de hacer mediante comprobaciones gráficas. Sin embargo, cuando se trata con múltiples dimensiones el proceso se complica.

Por ejemplo, considera un hombre que mide 2 metros y pesa 50 kg. Ninguno de los dos valores es atípico de forma individual, pero en conjunto se trataría de un caso muy excepcional.

La distancia de Mahalanobis es una medida de distancia entre un punto y la media que se ajusta en función de la correlación entre dimensiones y que permite encontrar potenciales outliers en distribuciones multivariante.

# PCA: Número de componentes principales

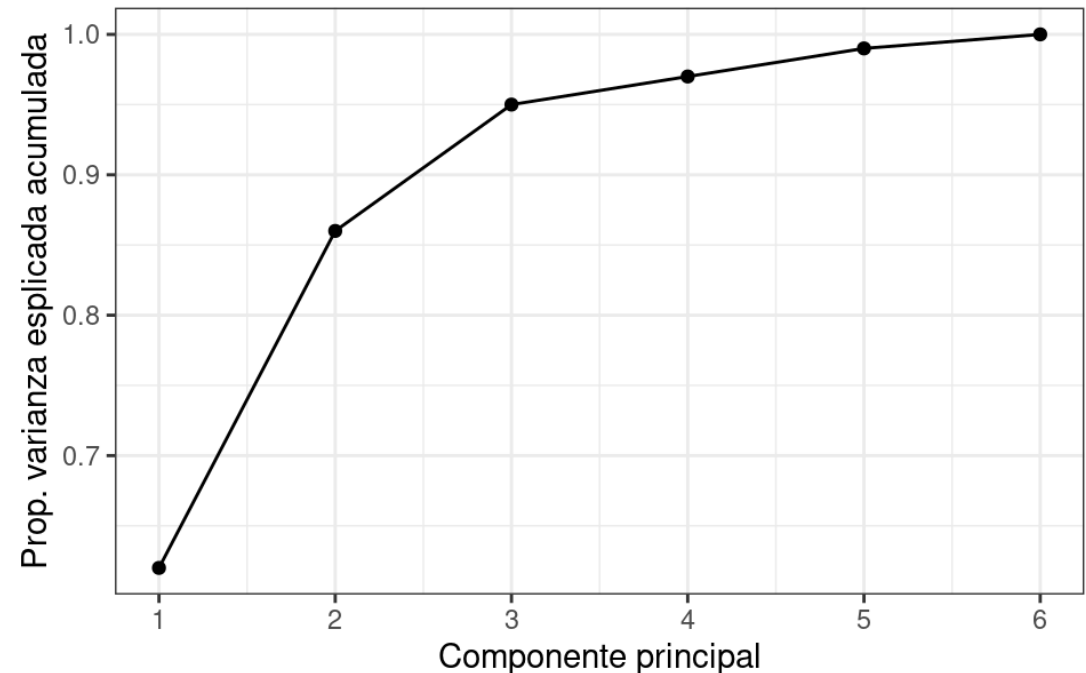
Un conjunto de datos con  $n$  observaciones y  $p$  variables el número de componentes principales distintas será:

$$\min(n-1, p)$$

*No existe un método objetivo para escoger el número de componentes principales que son suficientes para un análisis, por lo que depende del juicio del analista y del problema en cuestión.*

Si el objetivo es la visualización de los datos se suele tomar las **tres primeras componentes principales** siempre que expliquen con suficiente variabilidad los datos.

Si lo que se pretende es determinar el número de componentes que expliquen un mínimo porcentaje de varianza o que queramos utilizar para un análisis supervisado es mediante **validación cruzada**.



# Ejemplo en python

**Conjunto de datos:** *Clasificación de variedades de trigo*

**Información del conjunto de datos:**

El conjunto de datos comprendía granos de trigo pertenecientes a tres variedades diferentes de trigo: Kama, Rosa y Canadian, 70 elementos cada una.

Todos estos parámetros fueron de valor real continuo.

**Información de atributos:**

Para construir los datos, se midieron siete parámetros geométricos de los granos de trigo: área A, perímetro P, compacidad  $C = 4\pi A/P^2$ , longitud del grano, ancho del grano, coeficiente de asimetría y longitud del surco del grano.

# Ejemplo en python

```
import numpy as np
import pandas as pd
import time
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
target_names = {
    1:'Kama',
    2:'Rosa',
    3:'Canadian'
}
```

```
dataWheat = pd.read_csv("wheat.csv")
dataWheat['category'] = dataWheat['category'].map(target_names)
```

```
xWheat = dataWheat.drop('category', axis=1)
yWheat = dataWheat['category']
```

# Ejemplo en python

```
xWheatScaled = StandardScaler().fit_transform(xWheat)
```

```
sns.countplot( x='category', data=dataWheat)  
plt.title('Wheat targets value count')  
plt.show()
```

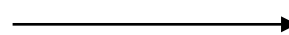
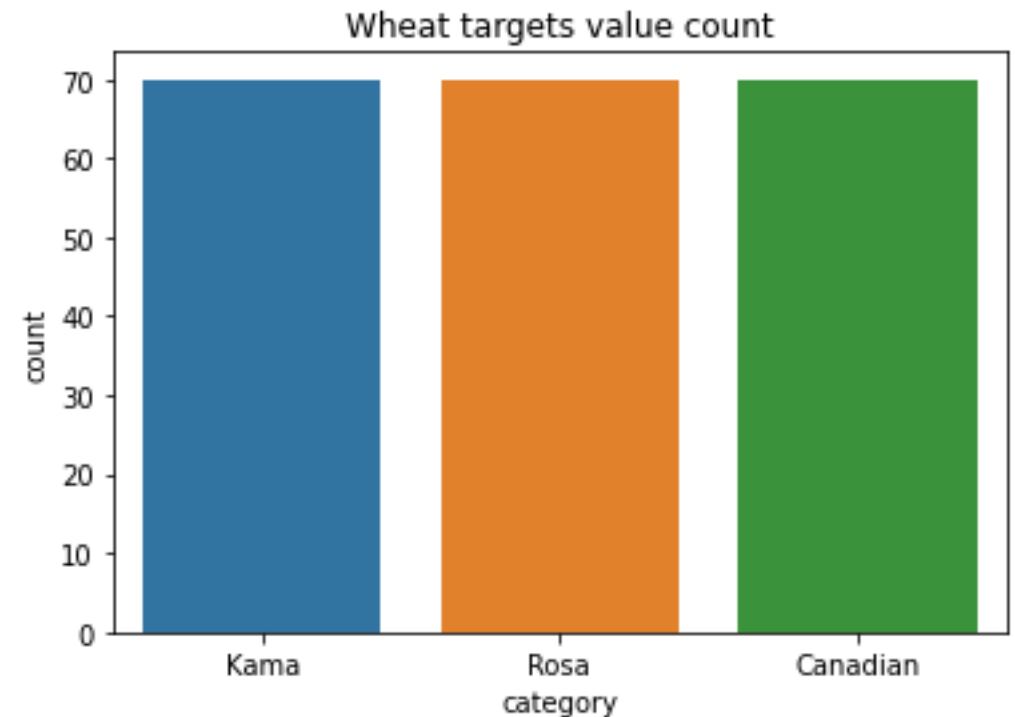
```
""" PCA """
```

```
pca = PCA(n_components=3)
```

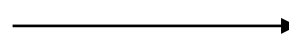
```
pcaFeatures = pca.fit_transform(xWheatScaled)
```

```
print('Shape before PCA: ', xWheatScaled.shape)
```

```
print('Shape after PCA: ', pcaFeatures.shape)
```



Shape before PCA: (210, 7)



Shape after PCA: (210, 3)

# Ejemplo en python

```
pcaWheat = pd.DataFrame(data=pcaFeatures, columns=['PC1', 'PC2', 'PC3'])
```

```
pcaWheat['category'] = yWheat.to_numpy()
```

pcaWheat



|     | PC1       | PC2       | PC3       | category |
|-----|-----------|-----------|-----------|----------|
| 0   | 0.317047  | 0.783669  | -0.631010 | Kama     |
| 1   | -0.003386 | 1.913214  | -0.669754 | Kama     |
| 2   | -0.459443 | 1.907225  | 0.932489  | Kama     |
| 3   | -0.591936 | 1.931069  | 0.499311  | Kama     |
| 4   | 1.102910  | 2.068090  | 0.056705  | Kama     |
| ... | ...       | ...       | ...       | ...      |
| 205 | -1.991107 | 0.865956  | 0.513303  | Canadian |
| 206 | -2.726865 | -0.208190 | -0.059059 | Canadian |
| 207 | -1.403633 | -1.298593 | 2.905811  | Canadian |
| 208 | -2.339328 | 0.099699  | -0.382515 | Canadian |
| 209 | -1.955953 | -0.525071 | 1.013113  | Canadian |

[210 rows x 4 columns]

# Ejemplo en python

pca.explained\_variance\_



array([5.05527392, 1.20330286, 0.68124747])

pca.singular\_values\_



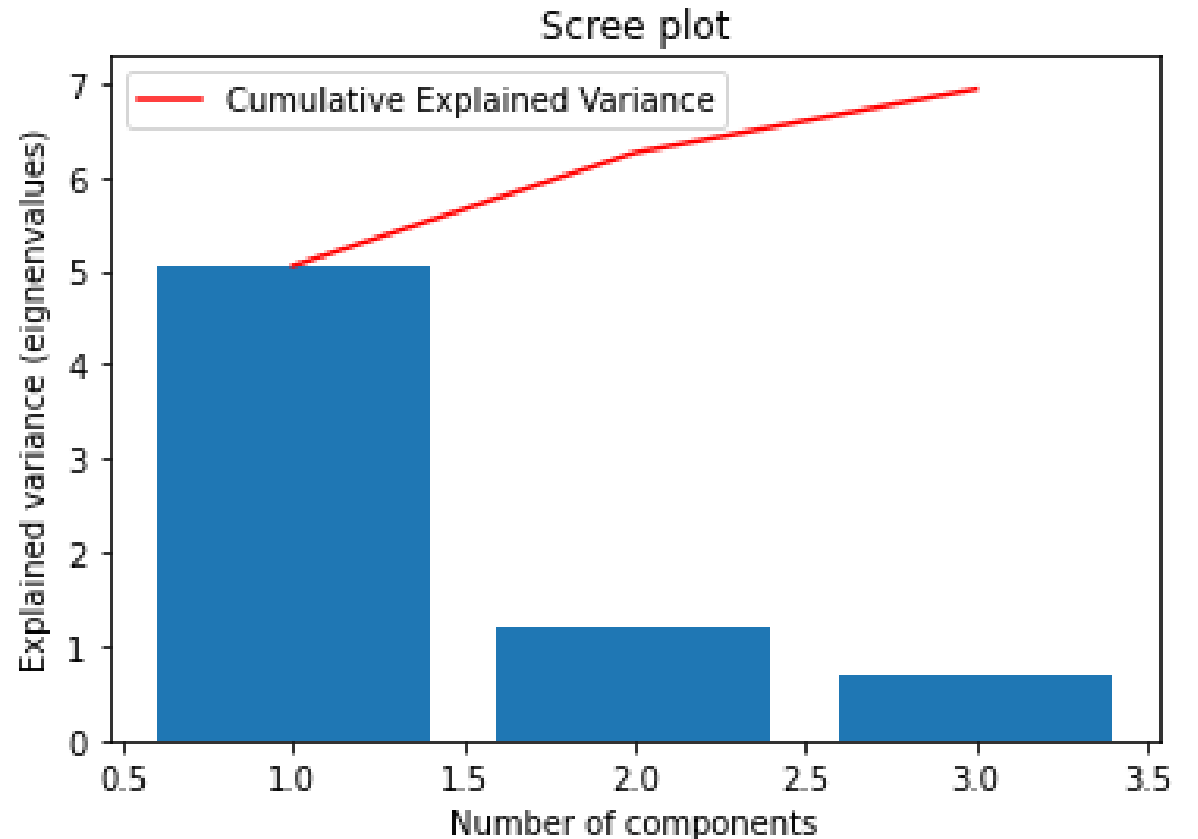
array([32.50464965, 15.85844563, 11.93233934])

```
# Bar plot of explained_variance
plt.bar(range(1,len(pca.explained_variance_)+1),
        pca.explained_variance_)

plt.plot(
    range(1,len(pca.explained_variance_)+1),
    np.cumsum(pca.explained_variance_),
    c='red', label='Cumulative Explained Variance')

plt.legend(loc='upper left')
plt.xlabel('Number of components')
plt.ylabel('Explained variance (eigenvalues)')
plt.title('Scree plot')

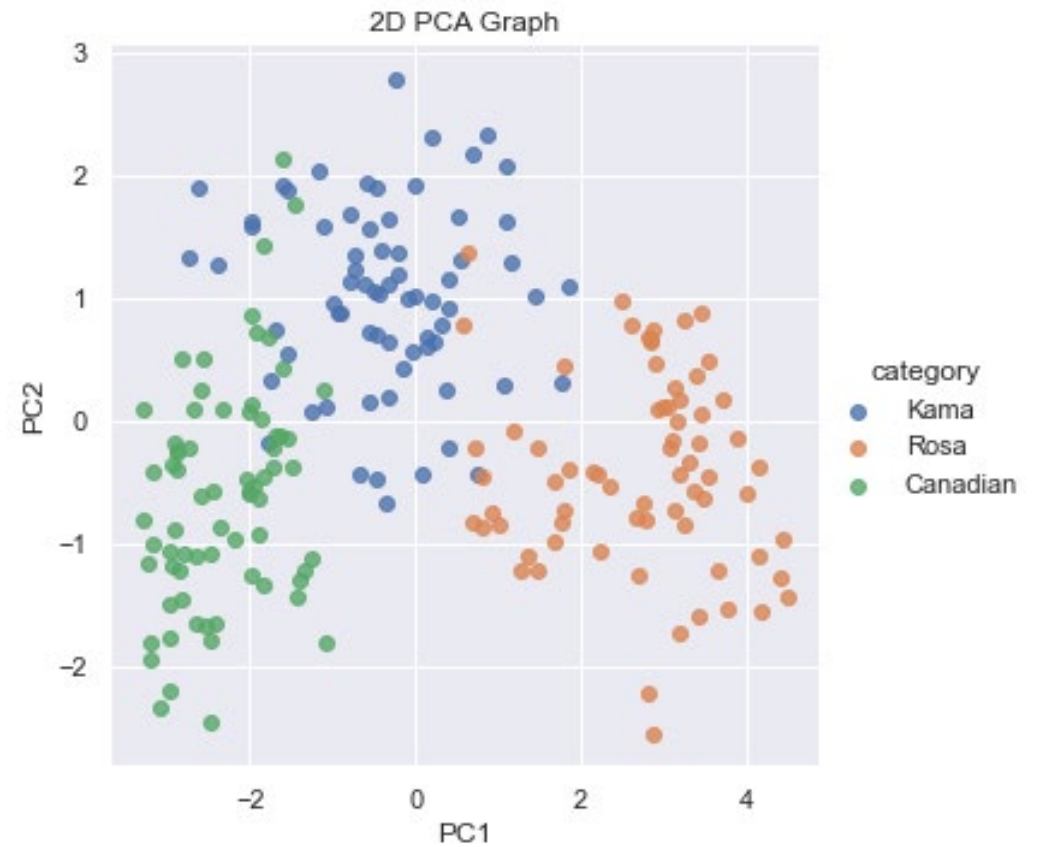
plt.show()
```



# Ejemplo en python

```
sns.set()
sns.lmplot(x='PC1', y='PC2', data=pcaWheat,
           hue='category', fit_reg=False, legend=True)

plt.title('2D PCA Graph')
plt.show()
```



# Escalado multi-dimensional (MDS)

Es un método para representar una matriz de disimilaridades en un cierto número de dimensiones. Se puede usar como una técnica de reducción de la dimensionalidad si se calculan las disimilaridades con respecto a la dimensión original y luego se representan en un espacio de menor dimensión.

La idea de MDS es que las distancias euclídeas en el espacio de llegada sean lo más cercanas posibles a las disimilaridades en el espacio original.

Si tenemos observaciones  $x_1, x_2, \dots, x_N$  y una función de disimilaridad  $d$ , la función de coste a optimizar en MDS podría ser:

$$C = \sum_{i \neq j} (d(x_i, x_j) - \|z_i - z_j\|)^2$$

donde  $z_i$  y  $z_j$  son los representantes en el espacio de llegada de  $x_i$  y  $x_j$  respectivamente.

Esta función puede optimizarse mediante descenso del gradiente

El descenso del gradiente es un algoritmo que encuentra un mínimo local en una función diferenciable.

Para encontrar un mínimo se mueve un punto candidato en el sentido contrario al gradiente (derivada) de manera proporcional a la magnitud del gradiente. Se itera este proceso hasta convergencia o hasta alcanzar un número de iteraciones prefijado.

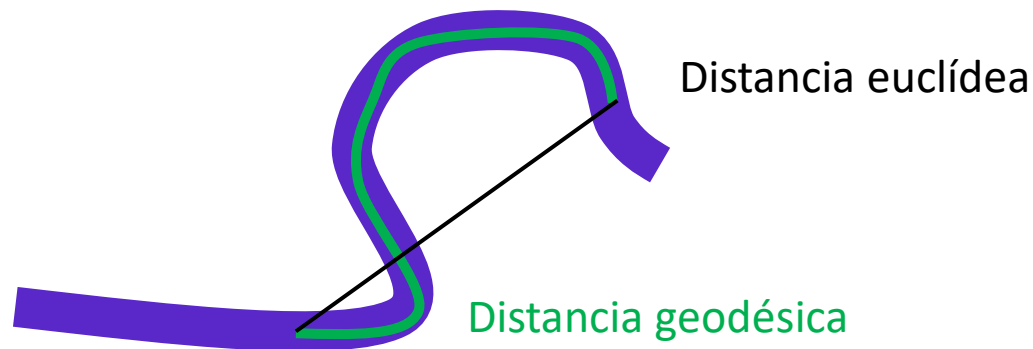
# ISOMAP

ISOMAP es método no paramétrico, lo cual significa que la técnica no especifica un mapeo directo hacia el espacio reducido. Los métodos no paramétricos tienen como desventaja que no es posible generalizarlos para nuevos datos sin realizar nuevamente el proceso de reducción de dimensión.

ISOMAP se basa en la idea de que muchos conjuntos de datos de alta dimensión en realidad tienen una estructura intrínseca de menor dimensión.

El método ISOMAP busca un espacio reducido embebido en el espacio original que mantenga las *distancias geodésicas* entre todos los puntos de coordenadas, con lo cual consiga caracterizar las vecindades presentes en la variedad. El uso de la distancia geodésica resulta mucho más expresivo y captura la distribución real de los datos.

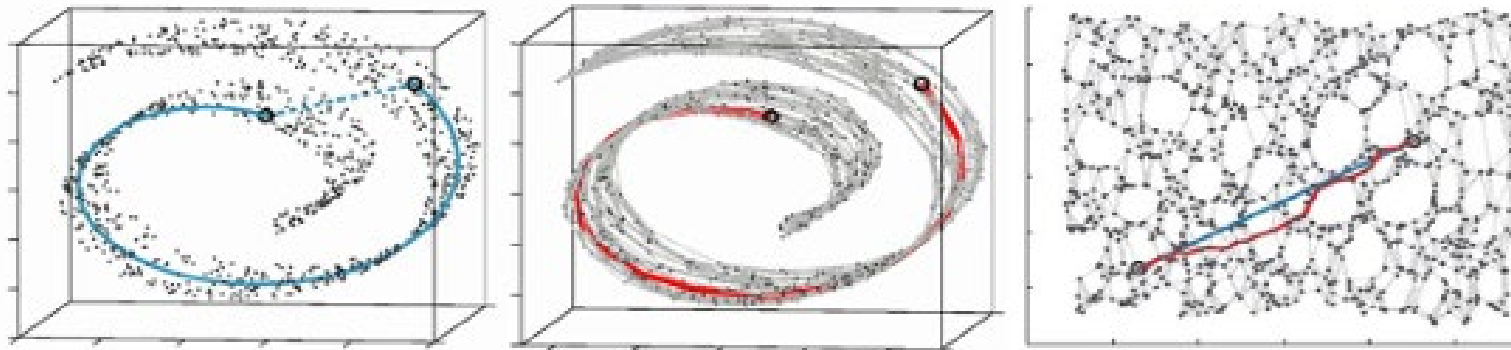
*Distancia geodésica:* Es la distancia de mínima longitud que une dos puntos en una superficie dada, y está contenida en esta superficie; por tanto, la línea más recta posible.



# Algoritmo ISOMAP

- En la primera se establecen las relaciones de vecindad existentes en el sub-espacio variedad  $\mathcal{M}$  basándose en las distancias Euclidianas  $d_X(i,j)$  obtenidas en el espacio original de entrada  $X$ . Las relaciones halladas se representan en un grafo ponderado  $\mathcal{G}$ , donde los pesos  $d_X(i,j)$  son asignados a los correspondientes bordes.
- Luego se estiman los pares  $d_{\mathcal{M}}(i,j)$  en  $\mathcal{M}$  y las distancias geodésicas son definidas como las menores distancias  $d_G(i,j)$  en  $\mathcal{G}$ .

Una vez que se dispone de las distancias geodésicas que preservan la naturaleza de los datos a través de las rutas geodésicas antes representadas con curvas en la variedad, estas se transforman en líneas rectas en la dimensión  $Y$ . ISOMAP realiza este proceso aplicando el método MDS a la matriz de distancias geodésicas  $D_G$ . MDS busca una representación Euclidiana, exacta o aproximada  $D_G$ , donde  $[D_G]_{ij}=d_G(i,j)$  en un espacio euclidiano  $Y$  de dimensión  $m$  que preserve la geometría intrínseca de  $\mathcal{M}$ .



# Algoritmo ISOMAP

## Entradas:

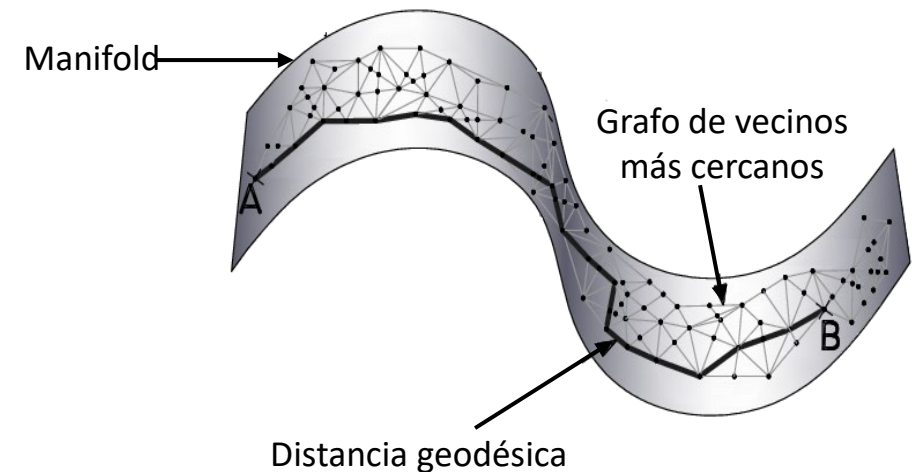
1. Un conjunto de datos  $X = \{x_1, x_2, \dots, x_n\}$  en un espacio de alta dimensión  $\mathbb{R}^d$
2. Parámetros: número de vecinos más cercanos ( $k$ )
3. La dimensionalidad deseada del espacio de salida ( $m$ )

## Pasos del algoritmo:

- a. Construcción del grafo de vecindad
- b. Cálculo de distancias geodésicas
- c. Aplicación de Escalamiento Multidimensional (MDS)

## Salida:

1. Una representación  $Y = \{y_1, y_2, \dots, y_n\}$  de los puntos originales en el espacio  $m$ -dimensional



# Algoritmo ISOMAP

## *Construcción del grafo de vecindad:*

1. Para cada punto  $x_i$ , se encuentra los  $k$  vecinos más cercanos.
2. Se crea un grafo  $G$  donde los nodos son los puntos de datos y se los conecta con sus  $k$  vecinos más cercanos.
3. Se les asignan pesos a las aristas basados en la distancia euclidiana entre los puntos.

## *Cálculo de distancias geodésicas:*

1. Se inicializa una matriz de distancia  $D$  del mismo tamaño que el número de puntos.
2. Para cada par de puntos  $(i, j)$ :
  - Si  $i$  y  $j$  son vecinos,  $D[i, j] = \text{distancia euclidiana entre ellos}$
  - Si no son vecinos,  $D[i, j] = \infty$
3. Se aplica el algoritmo de Floyd-Warshall o Dijkstra para encontrar las distancias de camino más corto entre todos los pares de puntos en el grafo. Estas distancias de camino más corto son las aproximaciones de las distancias geodésicas.

## *Aplicación de Escalamiento Multidimensional (MDS):*

1. Aplica MDS clásico a la matriz de distancias geodésicas  $D$ .
2. MDS encuentra una configuración de puntos en un espacio  $m$ -dimensional que mejor preserva las distancias en  $D$ .

# Algoritmo ISOMAP

## *Cálculo de vecinos más cercanos:*

Puede usar estructuras de datos como árboles k-d para una búsqueda eficiente.

## *Manejo de desconexiones:*

Si el grafo resultante no es totalmente conectado, ISOMAP puede funcionar en cada componente conectado por separado.

## *Optimización de MDS:*

La implementación exacta de MDS puede ser computacionalmente costosa para grandes conjuntos de datos. Técnicas de aproximación o métodos de optimización numérica pueden usarse para acelerar este paso.

## *Variantes y mejoras:*

1. *ISOMAP con puntos de referencia:* Usa un subconjunto de puntos como "puntos de referencia" para reducir la complejidad computacional.
2. *ISOMAP conformal:* Incorpora información sobre ángulos locales para mejorar la preservación de la estructura local.
3. *L-ISOMAP:* Una versión que utiliza distancias geodésicas locales para mejorar el rendimiento en ciertos tipos de datos.

# ISOMAP: Complejidad computacional y Limitaciones

## *Complejidad computacional:*

- La construcción del grafo de vecindad es  $O(n^2 \log n)$  si se usa un algoritmo de búsqueda de vecinos más cercanos basado en árboles.
- El cálculo de las distancias geodésicas usando Floyd-Warshall es  $O(n^3)$ .
- MDS clásico es  $O(n^3)$ .

Esto hace que ISOMAP sea computacionalmente intensivo para grandes conjuntos de datos, lo que ha llevado al desarrollo de versiones aproximadas y optimizadas.

## *Limitaciones:*

- Sensible al ruido en los datos.
- Computacionalmente intensivo para grandes conjuntos de datos.
- Asume que los datos yacen en un único Manifold conectado.

## *Interpretación de resultados:*

- Los ejes en el espacio reducido no tienen un significado directo como en PCA.
- La interpretación suele requerir conocimiento del dominio y análisis adicional

# ISOMAP: Ventajas vs Desventajas

## *Ventajas:*

- Captura relaciones no lineales que métodos lineales como PCA podrían perder.
- Preserva la estructura global de los datos.
- Puede revelar estructuras ocultas en datos complejos.

## *Desventajas:*

- No es posible delimitar cuanta información de la dimensión original es retenida en el espacio reducido al reconstruir los datos desde la dimensión reducida y midiendo el error entre los datos originales y los datos reconstruidos.
- Si bien la distancia geodésica podría resultar útil para añadir expresividad a los datos y explorar mayor información en conjuntos de datos complejos; ISOMAP tiene desventajas, entre ellas, la inestabilidad topológica (Balasubramanian 2010) que provoca que construya conexiones erróneas en el grafo de vecindad, lo que podría afectar su desempeño, la presencia de espacios "vacíos" en la variedad, o la susceptibilidad a variedades no convexas.

# ISOMAP - Consideraciones prácticas

## *Elección de $k$ :*

Un  $k$  muy pequeño puede resultar en un grafo desconectado, mientras que un  $k$  muy grande puede crear "atajos" que no representan la verdadera estructura del manifold.

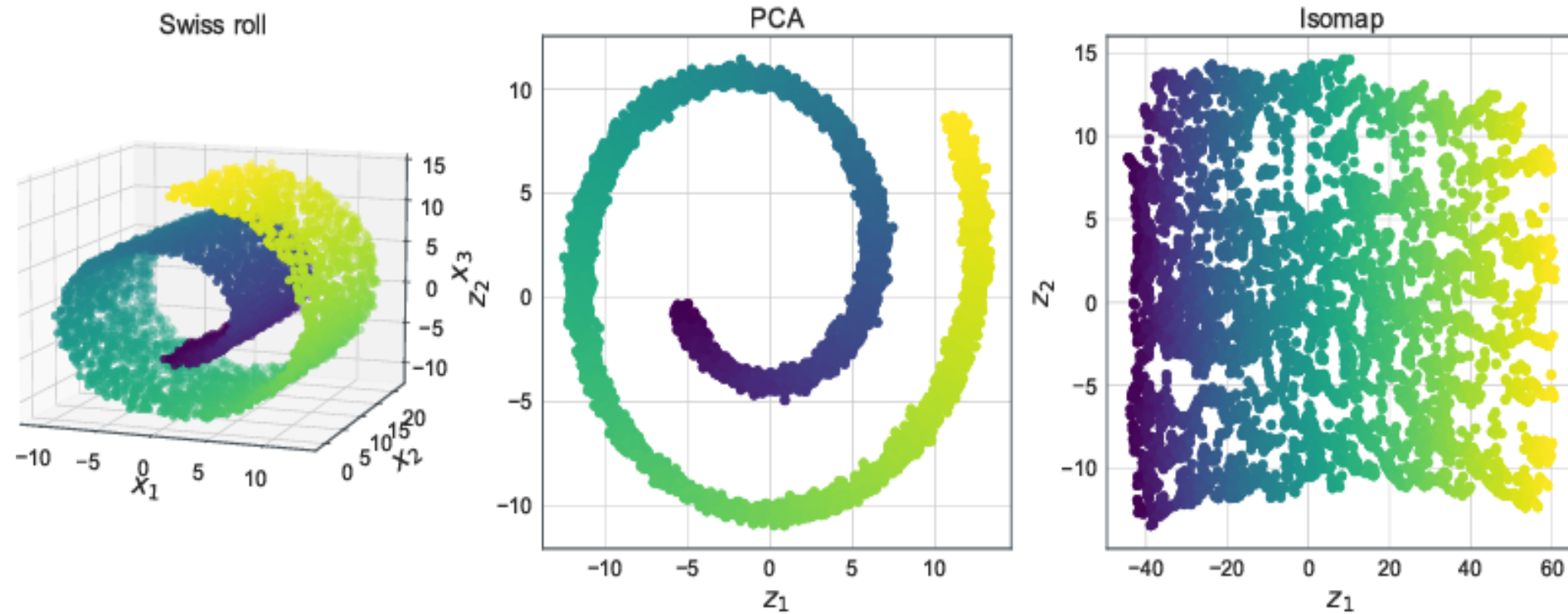
## *Normalización de datos:*

A menudo es beneficioso normalizar los datos antes de aplicar ISOMAP para evitar que características con grandes magnitudes dominen el análisis.

## *Evaluación:*

La calidad del embedding resultante puede evaluarse comparando las distancias en el espacio original con las distancias en el espacio reducido.

# ISOMAP vs PCA



Isomap es capaz de encontrar el manifold de los datos mientras que PCA, al ser un método lineal, no puede determinarlo

# ISOMAP - Resumen

El método ISOMAP establece relaciones de vecindad en la variedad basado en evaluaciones de las distancias geodésicas en las entradas y luego busca una representación Euclidiana, exacta o aproximada, que coincida con las evaluaciones geodésicas previas.

ISOMAP comienza estimando las distancias geodésicas entre los puntos en las entradas utilizando las distancias más cercanas en el grafo de los vecinos más cercanos del conjunto de datos. Para ello, construye un grafo ponderado de los vecinos más cercanos utilizando la distancia Euclidiana y lo recorre utilizando un algoritmo para calcular el camino mínimo (Dijkstra o Ford), produciendo como salida las distancias geodésicas.

Por ejemplo, aunque una imagen facial puede tener miles de píxeles (dimensiones), las variaciones reales pueden describirse con muchas menos dimensiones (pose, expresión, identidad, etc.).

## *Aplicaciones:*

- Visualización de datos de alta dimensión.
- Reconocimiento de patrones en imágenes y voz.
- Análisis de datos biomédicos.
- Procesamiento de lenguaje natural.

# Ejemplo en python

```
from sklearn.manifold import Isomap
```

```
""" Isomap """
```

```
isomapWheat = Isomap(n_neighbors=6, n_components=2)
```

```
isomapWheat.fit(xWheatScaled)
```

```
manifold_2Da = isomapWheat.transform(xWheatScaled)
```

```
manifold_2D = pd.DataFrame(manifold_2Da,  
    columns=['Component 1', 'Component 2'])
```

```
manifold_2D['category'] = yWheat.to_numpy()
```

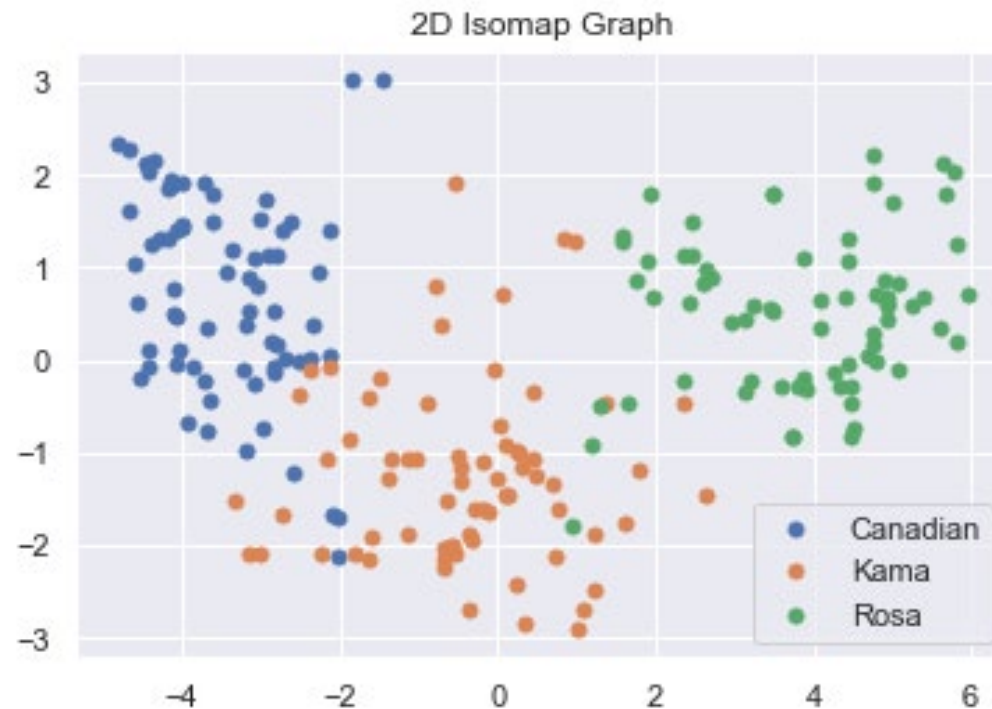
```
manifold_2D.head()
```



|   | Component 1 | Component 2 | category |
|---|-------------|-------------|----------|
| 0 | 0.460447    | -1.074534   | Kama     |
| 1 | 0.238061    | -2.425351   | Kama     |
| 2 | -0.637708   | -2.017595   | Kama     |
| 3 | -0.676373   | -2.146852   | Kama     |
| 4 | 1.243537    | -2.479073   | Kama     |

# Ejemplo en python

```
groups = manifold_2D.groupby('category')
plt.title('2D Isomap Graph')
for name, group in groups:
    plt.plot(group['Component 1'], group['Component 2'], marker='o', linestyle="", markersize=5, label=name)
plt.legend()
```



# t-SNE (t-distributed Stochastic Neighbor Embedding)

Es una técnica no lineal no supervisada utilizada principalmente para la exploración de datos y la visualización de datos de alta dimensión.

t-SNE se basa en la idea de que los puntos similares en el espacio de alta dimensión deberían estar cerca en el espacio de baja dimensión utilizando probabilidades para representar estas similitudes.

Stochastic Neighbor Embedding fue desarrollado y publicado por primera vez en 2002 por Hinton et al, y luego modificado por Laurens van der Maatens y Geoffrey Hinton en 2008 y llamado t-distributed Stochastic Neighbor Embedding (t-SNE).

t-SNE tiene como objetivos:

1. Preservación de la estructura local
2. Adaptación no lineal
3. Visualización de clusters
4. Balance entre estructura local y global

# Algoritmo t-SNE

## Entradas:

1. Un conjunto de datos  $X = \{x_1, x_2, \dots, x_n\}$  en un espacio de alta dimensión  $\mathbb{R}^d$
2. La dimensionalidad deseada del espacio de salida (generalmente 2 o 3)
3. Parámetros: perplejidad, tasa de aprendizaje, número de iteraciones

## Pasos del algoritmo:

- a. Cálculo de probabilidades en el espacio de alta dimensión
- b. Inicialización del espacio de baja dimensión
- c. Cálculo de probabilidades en el espacio de baja dimensión
- d. Optimización
- e. Iteración

## Salida:

1. Una representación  $Y = \{y_1, y_2, \dots, y_n\}$  de los puntos originales en el espacio m-dimensional

# Algoritmo t-SNE

*Cálculo de probabilidades en el espacio de alta dimensión:*

1. Para cada par de puntos  $x_i$  y  $x_j$ , se calcula la probabilidad condicional  $p(j|i)$ :

$$p(j|i) = \frac{e^{(-\|x_i - x_j\|^2 / 2\sigma_i^2)}}{\sum_{k \neq i} e^{(-\|x_i - x_k\|^2 / 2\sigma_i^2)}}$$

$\sigma_i$  se ajusta para cada punto para lograr la perplejidad deseada

***El rango normal de perplejidad está entre 5 y 50.***

2. Calcula la probabilidad conjunta simétrica:

$$P_{ij} = \frac{p(j|i) + p(i|j)}{2n}$$

*Inicialización del espacio de baja dimensión:*

- Se inicializa los puntos  $Y = \{y_1, y_2, \dots, y_n\}$  aleatoriamente en el espacio de baja dimensión usando una distribución normal o uniforme.

# Algoritmo t-SNE

## *Cálculo de probabilidades en el espacio de alta dimensión:*

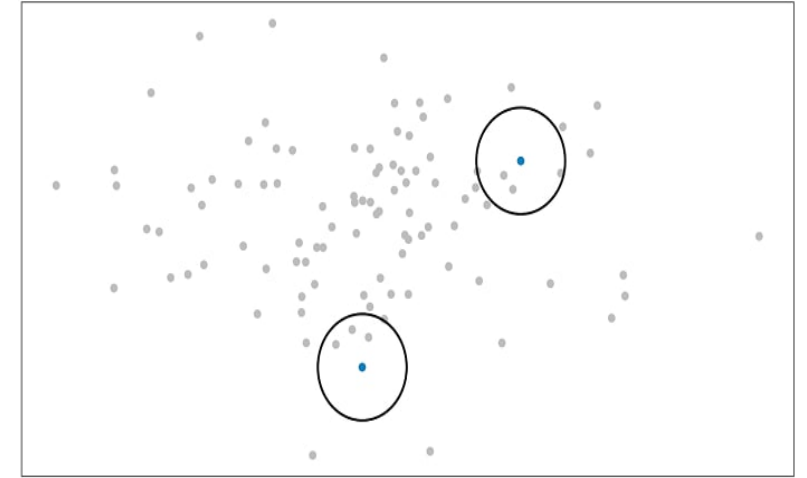
Supongamos un conjunto de datos dispersos en el plano. Para cada dato ( $x_i$ ) se centra una distribución gaussiana y luego se mide la densidad de todos los puntos ( $x_j$ ) bajo esa distribución gaussiana.

Esto da un conjunto de probabilidades ( $p_{ij}$ ) para todos los puntos.

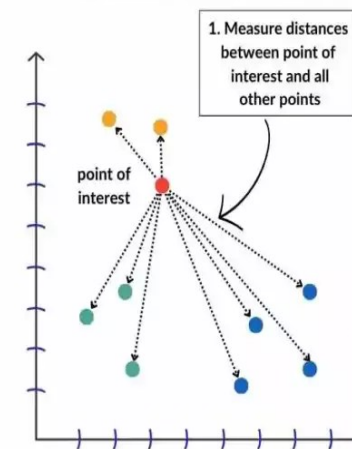
Esto significa es que si los puntos  $x_1$  y  $x_2$  tienen valores iguales bajo este círculo gaussiano, entonces sus proporciones y similitudes son iguales y, por lo tanto, tienes similitudes locales en la estructura de este espacio de alta dimensión.

La distribución gaussiana, círculo, se pueden manipular usando lo que se llama perplejidad, que influye en la varianza de la distribución (tamaño del círculo) y esencialmente en el número de vecinos más cercanos.

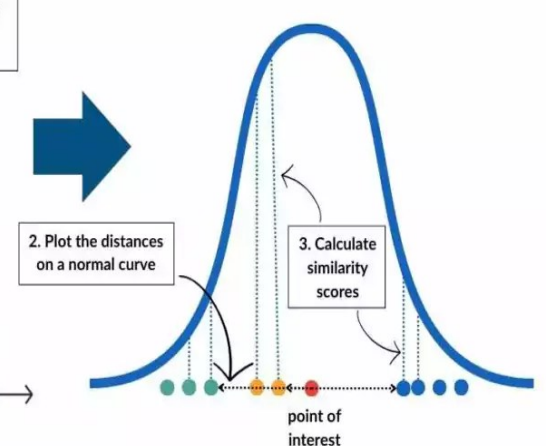
Gaussian Distribution Around Data Point



Original Multi-Dimensional Space



Normal Distribution Used To Calculate Similarity



# Algoritmo t-SNE

## *Cálculo de probabilidades en el espacio de baja dimensión:*

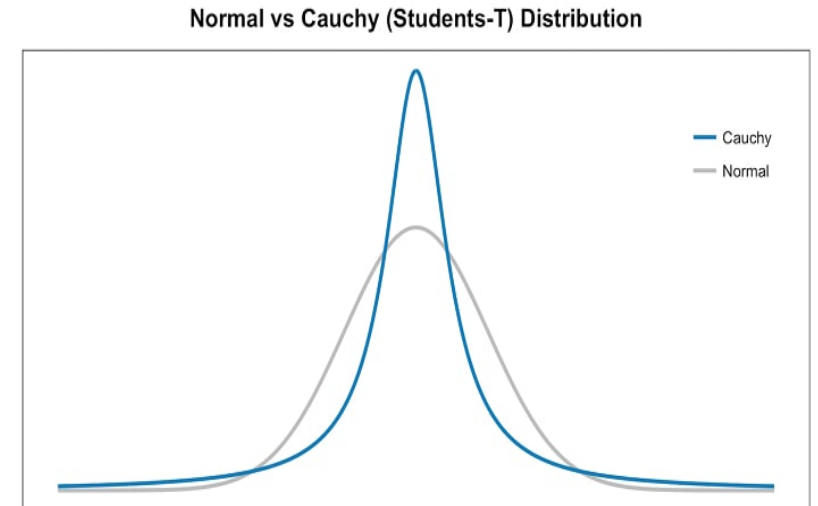
- Se calcula las probabilidades  $q_{ij}$  usando la distribución t-Student:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

Aquí se obtiene un segundo conjunto de probabilidades ( $q_{ij}$ ) pero en el espacio de baja dimensión.

Esto similar al primer paso, pero en lugar de usar una distribución gaussiana se usa una distribución t-Student con un grado de libertad, conocida como distribución de Cauchy.

La distribución t-Student tiene colas más pesadas que la distribución normal permitiendo un mejor modelado de distancias muy separadas.



# Algoritmo t-SNE

## *Optimización:*

- Se minimiza la divergencia de Kullback-Leibler entre P y Q:

$$KL(P||Q) = \sum_i \sum_j p_{ij} \log(p_{ij}/q_{ij})$$

usando el descenso de gradiente para actualizar las posiciones en el espacio de baja dimensión:

$$y_i = y_i - \eta \frac{\partial C}{\partial y_i}$$

donde  $\eta$  es la tasa de aprendizaje y C es el costo (KL divergencia).

## *Iteración:*

- Se Repite los pasos: *Cálculo de probabilidades en el espacio de baja dimensión* y *Optimización* por un número fijo de iteraciones o hasta la convergencia.

# Algoritmo t-SNE

## *Cálculo eficiente de gradientes:*

Se pueden usar trucos para calcular los gradientes de manera más eficiente.

## *Early exaggeration:*

En las primeras iteraciones, las probabilidades  $p_{ij}$  se multiplican por un factor (típicamente 4-12) para fomentar la formación de clusters.

## *Momentum:*

Se puede aplicar momentum al descenso de gradiente para acelerar la convergencia.

## *Variantes y mejoras:*

1. Barnes-Hut t-SNE: Utiliza aproximaciones de árbol para reducir la complejidad computacional de  $O(n^2)$  a  $O(n \log n)$ .
2. Fit-SNE (Fast Interpolation-based t-SNE): Usa interpolación y la transformada rápida de Fourier para acelerar los cálculos.
3. Multiscale t-SNE: Aplica t-SNE en múltiples escalas para capturar tanto estructuras locales como globales.

# t-SNE: Complejidad computacional y Limitaciones

## *Complejidad computacional:*

- La versión estándar de t-SNE tiene una complejidad de  $O(n^2)$  por iteración.
- Barnes-Hut t-SNE reduce esto a  $O(n \log n)$  por iteración.
- El número total de iteraciones suele ser del orden de 1000.

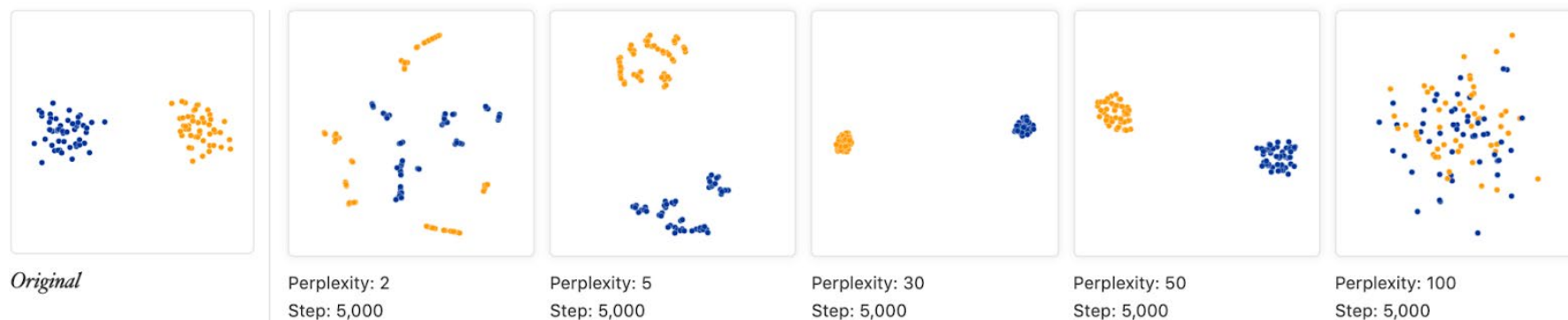
## *Limitaciones:*

- Computacionalmente intensivo para grandes conjuntos de datos.
- Los resultados pueden variar entre ejecuciones debido a la inicialización aleatoria.
- Puede ser difícil interpretar las distancias entre clusters lejanos.

## *Interpretación de resultados:*

- Los ejes en el espacio reducido no tienen un significado intrínseco.
- La distancia entre clusters puede no reflejar con precisión las diferencias en el espacio original.
- Es importante ejecutar t-SNE varias veces para asegurar la consistencia de los patrones observados.

# Algoritmo t-SNE: Perplejidad



La definición de perplejidad de Van der Maaten y Hinton puede interpretarse como una medida suave del número efectivo de vecinos.

El rendimiento de t-SNE es bastante robusto a los cambios en la perplejidad, y los valores típicos están entre 5 y 50.

Con valores de perplejidad en el rango (5 – 50), los diagramas muestran estos clusters aunque con formas muy diferentes.

- Para 2 dominan las variaciones locales.
- Para 100 los clusters se fusionan.

***Para que el algoritmo funcione correctamente, la perplejidad debe ser realmente menor que el número de puntos. De lo contrario, las implementaciones pueden tener un comportamiento inesperado.***

# Algoritmo t-SNE: Iteraciones



Las cuatro primeras se detuvieron antes de la estabilidad.

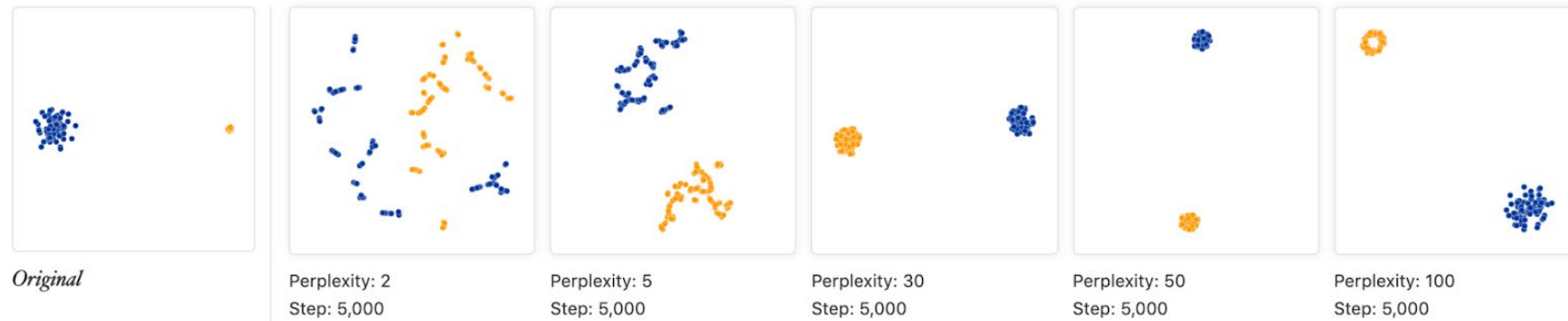
Si ve un gráfico de t-SNE con extrañas formas «pellizcadas», lo más probable es que el proceso se haya detenido demasiado pronto.

No hay un número fijo de pasos que produzca un resultado estable. Diferentes conjuntos de datos pueden requerir diferentes números de iteraciones para converger.

Otra pregunta natural es si diferentes ejecuciones con los mismos hiperparámetros producen los mismos resultados.

*En este sencillo ejemplo de dos clústeres múltiples ejecuciones dan la misma forma global. Sin embargo, algunos conjuntos de datos producen diagramas muy diferentes en diferentes ejecuciones.*

# t-SNE: Agrupaciones con diferente interdistancia



Los dos conglomerados parecen tener el mismo tamaño en los gráficos de t-SNE. *¿Qué ocurre?*

El algoritmo t-SNE adapta su noción de «distancia» a las variaciones regionales de densidad en el conjunto de datos. Como resultado, amplía de forma natural los clusters densos y contrae los escasos, igualando el tamaño de los clusters.

Se trata de un efecto diferente al hecho habitual de que cualquier técnica de reducción de la dimensionalidad porque las distancias están distorsionadas.

El resultado final es que no se pueden ver los tamaños relativos de los clusters en un gráfico t-SNE.

# t-SNE: Consideraciones prácticas y Aplicaciones

## *Consideraciones prácticas:*

1. Elección de la perplejidad: Valores típicos están entre 5 y 50. Es recomendable probar varios valores.
2. Normalización de datos: Es importante normalizar o estandarizar los datos antes de aplicar t-SNE.
3. Múltiples ejecuciones: Debido a la inicialización aleatoria, es buena práctica ejecutar t-SNE varias veces y comparar los resultados.
4. Evaluación: No hay una métrica única para evaluar la calidad de los resultados de t-SNE. La inspección visual y la coherencia con el conocimiento del dominio son importantes.

## *Aplicaciones:*

- Visualización de datos genómicos.
- Análisis de imágenes de alta dimensión.
- Exploración de datos de redes sociales.
- Visualización de embeddings de palabras en NLP.

# Ejemplo en python

```
from sklearn.manifold import TSNE
```

```
""" t-SNE """
```

```
time_start = time.time()
```

```
tsneWheat = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=300)
```

```
tsneWheatResults = tsneWheat.fit_transform(xWheatScaled)
```

```
print('t-SNE done! Time elapsed: {} seconds'.format(time.time()-time_start))
```

```
[t-SNE] Computing 121 nearest neighbors...
```

```
[t-SNE] Indexed 210 samples in 0.000s...
```

```
[t-SNE] Computed neighbors for 210 samples in 0.135s...
```

```
[t-SNE] Computed conditional probabilities for sample 210 / 210
```

```
[t-SNE] Mean sigma: 1.044265
```

```
[t-SNE] KL divergence after 250 iterations with early
```

```
exaggeration: 45.464500
```

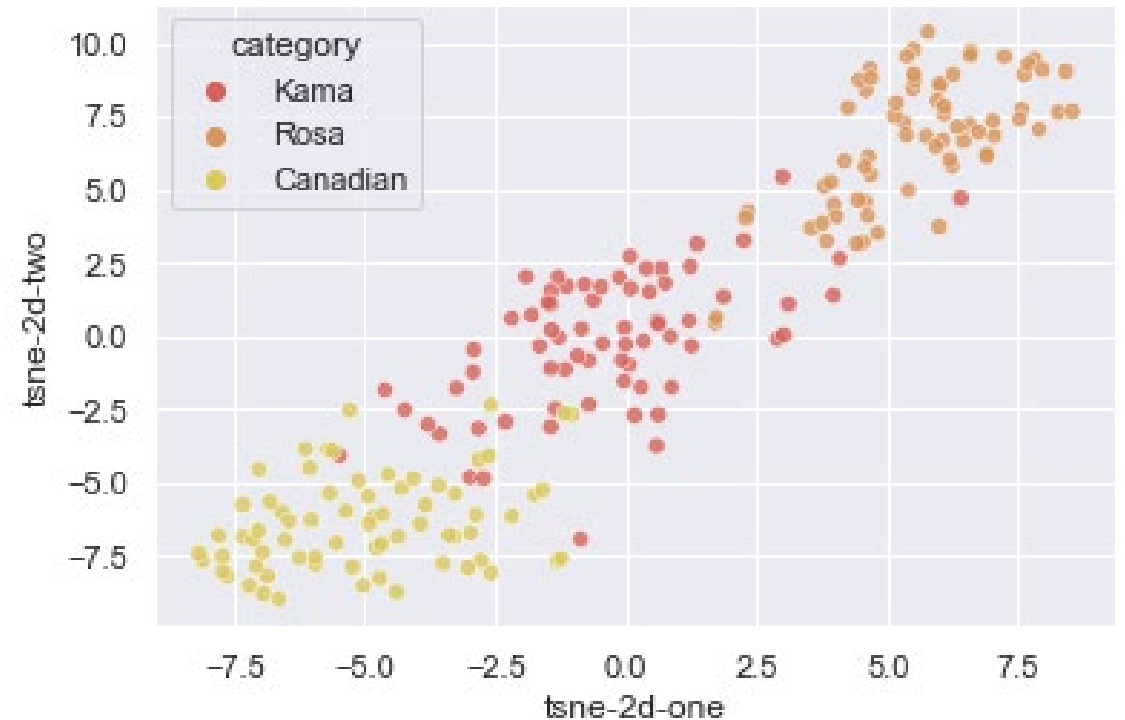
```
[t-SNE] KL divergence after 300 iterations: 0.288758
```

```
t-SNE done! Time elapsed: 0.3541288375854492 seconds
```

# Ejemplo en python

```
subsetWheatTSNE = pd.DataFrame(yWheat)
subsetWheatTSNE['tsne-2d-one'] = tsneWheatResults[:,0]
subsetWheatTSNE['tsne-2d-two'] = tsneWheatResults[:,1]
```

```
plt.figure(figsize=(16,10))
sns.scatterplot(
    x="tsne-2d-one", y="tsne-2d-two",
    hue="category",
    palette=sns.color_palette("hls", 15),
    data=subsetWheatTSNE,
    legend="full", alpha=0.8)
```

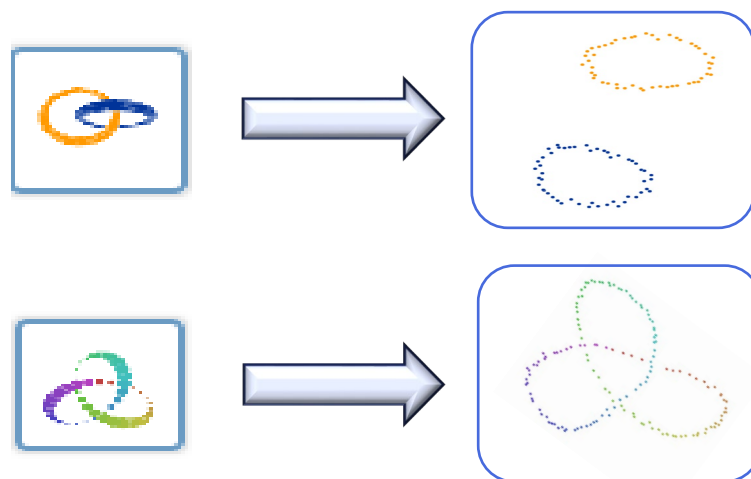


# UMAP (Uniform Manifold Approximation and Projection)

La idea básica detrás de UMAP radica en construir *una representación de baja dimensión de los datos que preserve la estructura global y local del espacio de alta dimensión*.

Este algoritmo combina ideas de la teoría de manifolds, topología y aprendizaje automático para proporcionar una poderosa herramienta de reducción de dimensionalidad y visualización.

UMAP utiliza un enfoque basado en gráficos para construir una representación topológica de los datos, que luego se incrusta en un espacio de baja dimensión mediante un descenso de gradiente estocástico.



# Algoritmo UMAP

## Entradas:

1. Un conjunto de datos  $X = \{x_1, x_2, \dots, x_n\}$  en un espacio de alta dimensión  $\mathbb{R}^d$
2. La dimensionalidad deseada del espacio de salida (generalmente 2 o 3)
3. Parámetros: mínima distancia, métrica, número de vecinos, número de épocas, semilla

## Pasos del algoritmo:

- a. Construcción del grafo de vecinos más cercanos
- b. Representación topológica difusa
- c. Construcción del simplicial complex de baja dimensión
- d. Optimización
- e. Iteración

## Salida:

1. Una representación  $Y = \{y_1, y_2, \dots, y_n\}$  de los puntos originales en el espacio m-dimensional
2. Modelo UMAP entrenado (opcional)

# Algoritmo UMAP

## *Construcción del grafo de vecinos más cercanos:*

1. Para cada punto  $x$  en el conjunto de datos se buscan los  $k$  vecinos más cercanos de  $y$  y se calcula la distancia al vecino más lejano ( $\rho$ ).
2. Se construye un grafo dirigido conectando cada punto a sus  $k$  vecinos más cercanos.

## *Representación topológica difusa:*

1. Para cada par de puntos  $(x, y)$  conectados en el grafo se calcula la probabilidad de conexión  $p$ :

$$p = e^{-\frac{d(x,y)-\rho}{\sigma}}$$

donde  $d(x,y)$  es la distancia entre  $x$  e  $y$ , y  $\sigma$  es un factor de escala local

2. Se simetriza el grafo tomando el máximo de las probabilidades en ambas direcciones.

## *Construcción del simplicial complex de baja dimensión:*

1. Inicializa los puntos aleatoriamente en el espacio de baja dimensión.
2. Para cada par de puntos  $(x, y)$  en el espacio de baja dimensión se calcula la probabilidad de conexión en el espacio de baja dimensión  $q$ :

$$q = (1 + a||x - y||^{2b})^{-1}$$

donde  $a$  y  $b$  son hiperparámetros

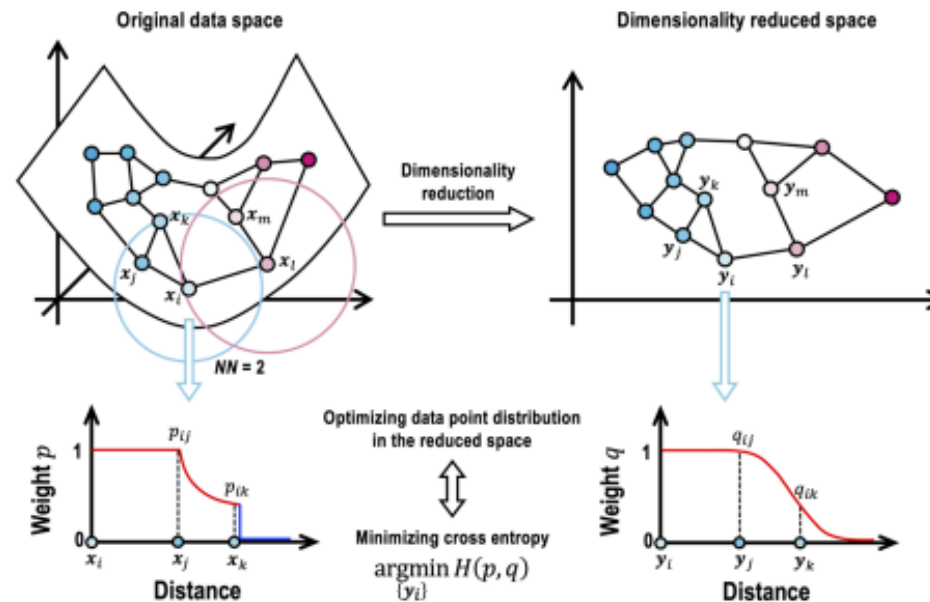
# Algoritmo UMAP

## Optimización:

1. Se define una función de costo basada en la divergencia entre las representaciones de alta y baja dimensión.  
Por lo general la entropía cruzada  $H$ .
2. Se minimiza esta función de costo usando descenso de gradiente estocástico.
3. Se actualizan las posiciones de los puntos en el espacio de baja dimensión.

## Iteración:

Se repite el paso de minimización en el proceso de Optimización por un número fijo de épocas o hasta que se alcance la convergencia.



# UMAP: Complejidad computacional, Limitaciones y Aplicaciones

## *Complejidad computacional:*

$O(n \log n)$ , donde  $n$  es el cantidad de datos.

## *Limitaciones:*

- Al igual que t-SNE, los resultados pueden variar entre ejecuciones.
- La interpretación de las distancias entre clusters lejanos puede ser engañosa.

## *Aplicaciones:*

- Visualización de datos de alta dimensión
- Reducción de dimensionalidad como paso de preprocesamiento
- Análisis de datos de single-cell RNA-seq en biología

UMAP es una herramienta de exploración y visualización. Los patrones observados deben ser validados con análisis estadísticos adicionales y conocimiento del dominio antes de sacar conclusiones definitivas.

# UMAP: Interpretación de los resultados

## *Visualización de clusters:*

Los puntos que forman grupos o clusters en el espacio de baja dimensión generalmente representan datos que son similares en el espacio original de alta dimensión.

La presencia de clusters distintos puede indicar subgrupos o categorías en tus datos.

## *Distancias y proximidad:*

Puntos que están cerca en el espacio de UMAP generalmente corresponden a puntos que son similares en el espacio original.

Sin embargo, es importante notar que las distancias absolutas en el espacio UMAP no siempre se correlacionan directamente con las distancias en el espacio original.

## *Forma global:*

La forma general de la distribución de puntos puede revelar estructuras en los datos, como manifolds no lineales o topologías complejas.

Patrones como curvas, espirales o formas más complejas pueden indicar relaciones no lineales en los datos originales.

# UMAP: Interpretación de los resultados

## *Continuidad y transiciones:*

Áreas donde los puntos forman un continuo pueden representar transiciones graduales entre diferentes estados o categorías en los datos originales.

## *Outliers:*

Puntos que aparecen aislados o muy separados del resto pueden representar outliers o casos atípicos en los datos originales.

## *Preservación de la estructura local vs. global:*

UMAP intenta preservar tanto la estructura local como la global. Observa si puedes identificar tanto agrupaciones locales como relaciones a mayor escala entre grupos.

## *Estabilidad de los resultados:*

Ejecuta UMAP múltiples veces con diferentes inicializaciones aleatorias. Estructuras que aparecen consistentemente son más propensas a reflejar características reales de los datos.

## *Limitaciones en la interpretación:*

UMAP es una proyección y puede no capturar toda la información de los datos originales.

Las distancias entre clusters lejanos pueden no ser proporcionales a las distancias en el espacio original.

# Ejemplo en python

```
from umap import UMAP
```

```
""" UMAP """
```

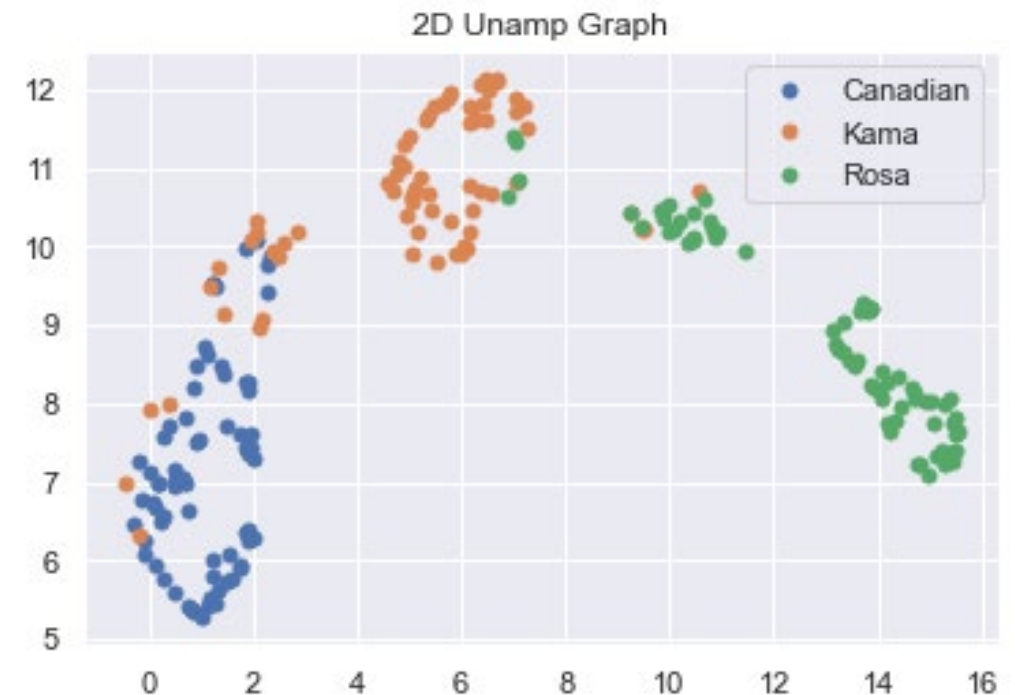
```
umap_2d = UMAP(n_components=2, init='random', random_state=0)
```

```
umap_3d = UMAP(n_components=3, init='random', random_state=0)
```

```
proj_2d = umap_2d.fit_transform(xWheat)
```

```
umapProj2D = pd.DataFrame(proj_2d,  
    columns=['Component 1', 'Component 2'])  
umapProj2D['category'] = yWheat.to_numpy()
```

```
groups = umapProj2D.groupby('category')  
plt.title('2D Unamp Graph')  
for name, group in groups:  
    plt.plot(group['Component 1'], group['Component 2'], marker='o',  
        linestyle="", markersize=5, label=name)  
plt.legend()
```



# Ejemplo en python

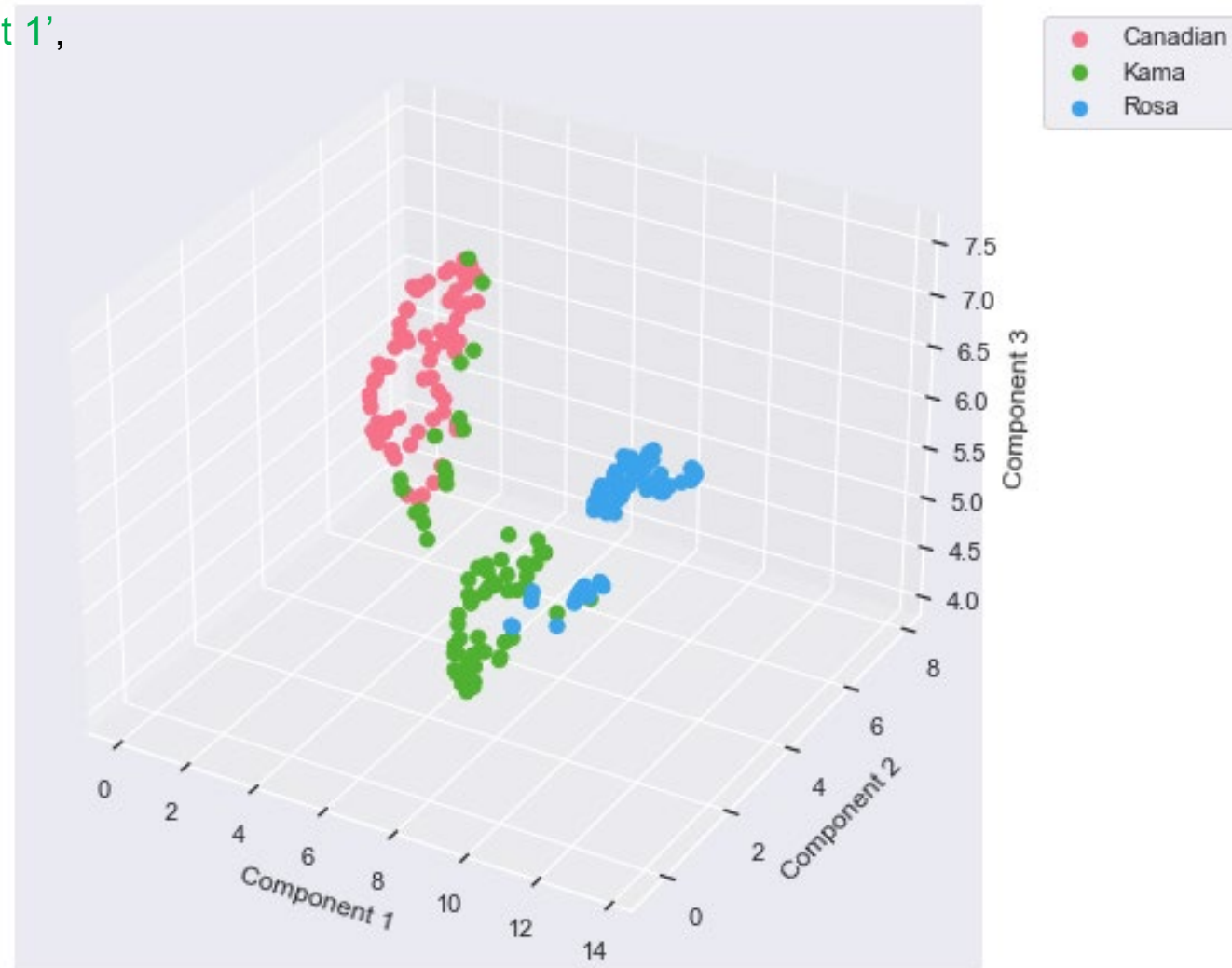
```
proj_3d = umap_3d.fit_transform(xWheat)
umapProj3D = pd.DataFrame(proj_3d, columns=['Component 1',
      'Component 2', 'Component 3'])
umapProj3D['category'] = yWheat.to_numpy()
```

```
fig = plt.figure(figsize=(10, 6))
ax = Axes3D(fig, auto_add_to_figure=False)
fig.add_axes(ax)
```

```
labels = np.unique(umapProj3D['category'])
palette = sns.color_palette("husl", len(labels))
```

```
for label, color in zip(labels, palette):
    df1 = umapProj3D[umapProj3D['category'] == label]
    ax.scatter(df1['Component 1'], df1['Component 2'],
              df1['Component 3'], s=40, marker='o',
              color=color, alpha=1, label=label)
ax.set_xlabel('Component 1')
ax.set_ylabel('Component 2')
ax.set_zlabel('Component 3')
```

```
plt.legend(bbox_to_anchor=(1.05, 1), loc=2)
plt.show()
```



# Comparación entre PCA, ISOMAP, t-SNE y UMAP

- 1. Lineal versus no lineal:** PCA es una técnica de reducción de dimensionalidad lineal que se enfoca en encontrar ejes ortogonales que expliquen la varianza máxima en los datos. Es eficaz para capturar patrones globales y reducir la dimensionalidad de los datos. Por el contrario, t-SNE y UMAP son técnicas no lineales que tienen como objetivo preservar las estructuras locales y globales en los datos, haciéndolas más adecuadas para visualizar y explorar relaciones complejas.
- 2. Preservación de la estructura global frente a la local:** PCA se centra principalmente en preservar la estructura global de los datos, lo que significa que su objetivo es capturar la variación y las tendencias generales en el conjunto de datos. Es posible que no preserve la estructura local detallada o los patrones de agrupación en los datos. Por otro lado, t-SNE y UMAP destacan por preservar la estructura local, enfatizando las relaciones entre puntos de datos cercanos. Pueden revelar grupos, patrones y relaciones no lineales que pueden estar ocultas en un espacio de alta dimensión.
- 3. Tiempo de cálculo y escalabilidad:** PCA es computacionalmente eficiente y puede manejar grandes conjuntos de datos. Es una técnica basada en álgebra lineal que se puede calcular de manera eficiente mediante operaciones matriciales. t-SNE y UMAP, por otro lado, requieren más procesamiento computacional, especialmente para grandes conjuntos de datos. Implican procedimientos de optimización y cálculos basados en gráficos, que pueden llevar mucho tiempo. UMAP está diseñado para ser más escalable que t-SNE y puede manejar conjuntos de datos a gran escala de manera más eficiente.

# Comparación entre PCA, ISOMAP, t-SNE y UMAP

4. **Interpretabilidad:** PCA proporciona resultados interpretables ya que los componentes principales representan combinaciones lineales de las características originales. Cada componente principal captura una cierta cantidad de varianza en los datos y sus contribuciones pueden analizarse. Por el contrario, t-SNE y UMAP producen incrustaciones transformadas en un espacio de dimensiones inferiores que no son directamente interpretables en términos de las características originales. Se utilizan principalmente con fines de visualización y exploración.
5. **Idoneidad para diferentes tareas:** PCA se usa comúnmente para reducción de dimensionalidad, selección de características y reducción de ruido. También se utiliza como paso de preprocesamiento para varios algoritmos de aprendizaje automático. t-SNE y UMAP se emplean a menudo para la visualización y exploración de datos. Ayudan a revelar estructuras, grupos y relaciones complejas en los datos, lo que los hace valiosos para tareas como el análisis exploratorio de datos, el reconocimiento de patrones y la detección de anomalías.
6. **Ajuste de parámetros:** PCA no tiene muchos parámetros ajustables. La cantidad de componentes principales a retener es una decisión clave, que puede basarse en la variación explicada o en el conocimiento del dominio. t-SNE y UMAP tienen más parámetros para ajustar. Por ejemplo, en t-SNE, la perplejidad, la tasa de aprendizaje y el número de iteraciones influyen en los resultados. UMAP tiene parámetros como `n_neighbors`, `min_dist` y `metric` que afectan la calidad de la incrustación.

# Comparación entre PCA, ISOMAP, t-SNE y UMAP en el MNIST dataset

